

Introduction

Course: 400521-004 컴퓨터프로그래밍 I · Computer Programming I

Instructor: Yushintia Pramitarini

Year / Semester: 2025 · Semester 1

Format: 15 meetings × 3 × 50 min · Language: C (C99)

Who This Manual Is For

This manual is for complete beginners. If you have never written a single line of code, you are exactly who we wrote it for. You do not need any prior background in programming, and you do not need to think of yourself as a “math person” or a “computer person” to succeed here.

Programming is a skill, and like any skill, it grows with practice. Every professional developer once stared at their first error message with no idea what it meant. This course walks you through that journey one small, achievable step at a time.

Our goal is not to race through every feature of the C language. It is to help you build real problem-solving confidence: the ability to look at a problem, break it into steps, and turn those steps into working code. That way of thinking is what you will carry forward into every advanced course and every program you ever write.

Why C?

C is a high-level language that stays close to the machine. Learning C teaches you:

- How variables and data are stored in memory
- How the CPU executes instructions one at a time
- Why bugs like buffer overflows and off-by-one errors exist

These insights prepare you for **System Programming**, **Computer Architecture**, and **Operating Systems**, which all build directly on what you learn here.

Course Objectives

By the end of this course you will be able to:

1. Understand the basic principles of programming and problem-solving.
2. Read and write syntactically correct C programs.
3. Use variables, data types, and expressions correctly.
4. Apply `if/else`, `switch`, `while`, `do-while`, and `for` control structures.
5. Design and call functions; organize code into reusable modules.
6. Declare, initialize, and traverse one-dimensional and two-dimensional arrays; handle C strings.

Course Journey: 15 Weeks at a Glance

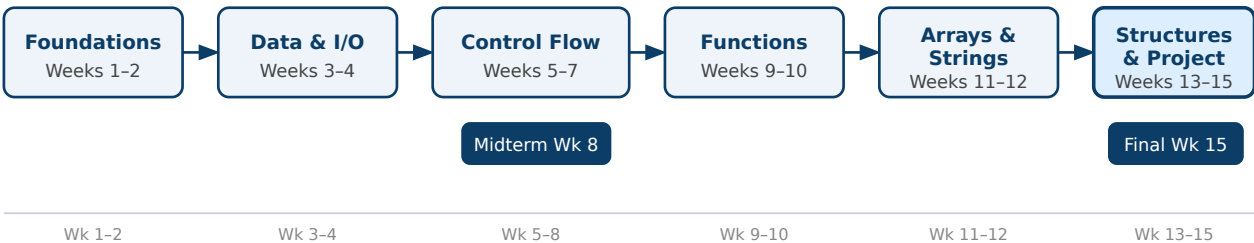


Figure 1.1. Course roadmap: six phases across 15 weeks.

Weeks	Theme	Objectives covered
1-2	Foundations: what programming is, first program	1-2
3-4	Data: variables, types, expressions, I/O	3
5-7	Control flow: conditionals, loops	4
8	Midterm (live coding, weeks 1-7)	1-4
9-10	Functions and modular design	5
11-12	Arrays and strings	6
13	Basic data structures (struct, intro pointers)	6 + awareness
14	Systematic debugging and project finalization	1-6
15	Final (project demo + individual oral defense)	1-6

How to Use a Lab

Each lab page follows the same ten-section template:

1. **Header:** week, topic, duration, prerequisites, method
2. **Learning outcomes:** what you will be able to do by the end
3. **Recap:** one paragraph connecting to the prior week
4. **Background:** concise theory (with optional *Under the Hood* box)
5. **Worked examples:** annotated programs with expected output
6. **Guided exercises:** type-along coding tasks (Period B)
7. **Challenge problems:** optional harder tasks
8. **Common pitfalls:** the errors everyone makes on this topic
9. **Submission and rubric:** what to submit, filename, points
10. **Further reading:** mapped textbook sections

Under the Hood boxes are short curiosity notes about how hardware relates to the concept you just learned. They are **never assessed**. They exist to build intuition, and they signpost deeper material you will study in later courses.

Assessment Summary

Component	Weight	When
Midterm (live in-lab coding)	20%	Week 8
Project (artifact: code + README)	30%	Submitted Week 14
Final (project demo + oral defense)	30%	Week 15
Attendance	10%	Ongoing
Assignments (weekly deliverables)	10%	Weeks 2–13

Full rubrics are in [Appendix: Grading Rubrics](#).

Academic Integrity and Collaboration Policy

- **Weekly assignments:** you may discuss ideas with classmates but must write your own code. Sharing source files or copying code is not permitted.
- **Take-home project:** the project is strictly individual. You must be able to explain every line in the Week-15 oral defense. Submitting code you did not write and cannot explain is an academic integrity violation.
- **Midterm:** closed collaboration, restricted network; open-book (compiler and reference sheets allowed).
- When in doubt, ask the instructor before submitting, not after.

Course Resources

- **Primary:** K. N. King, *C Programming: A Modern Approach*, 2nd ed.
- **Reference:** Kernighan and Ritchie, *The C Programming Language*, 2nd ed.
- **Online:** cppreference.com, OnlineGDB (browser C compiler)
- **Full reference list:** [Appendix: References](#)

Toolchain & Development Environment

Before you can run your first C program, you need three things: a **compiler**, a **text editor**, and a **terminal**.

In plain words: compiler A compiler is a program that reads your code and translates it into a language the computer can actually run. Think of it like a human translator who reads a document in English and produces a version in Japanese. Your C source code is the English; the compiled program is the Japanese. Without the translation, the computer cannot understand your instructions.

In plain words: text editor A text editor is the tool you use to write and save your code files. It is similar to a word processor (like Microsoft Word), but it saves files as plain text with no hidden formatting. You cannot use Word or Google Docs to write C code because they add invisible characters that the compiler cannot understand. A good code editor also colours your code (syntax highlighting) and warns you about mistakes before you even compile.

In plain words: terminal A terminal (also called a command prompt or shell) is a text window where you type commands. Instead of clicking buttons, you type `gcc hello.c -o hello` and press Enter. The computer runs the command and shows you the result as text. Compiling, running your program, and checking output all happen in the terminal.

Option A: Online Compiler (No Install Required)

If you are in the first session and your machine is not set up yet, use a browser-based compiler:

Site	URL
OnlineGDB	https://www.onlinegdb.com/online_c_compiler
Programiz	https://www.programiz.com/c-programming/online-compiler/

How to use OnlineGDB in three steps:

1. Open the URL in your browser.
2. Click the language selector (top-left) and choose **C**.

3. Type your code in the editor pane, then click the green **Run** button. Your output appears in the pane at the bottom.

This is a fallback; you will set up a local environment during Lab 01.

Option B: Local Setup

A local setup runs entirely on your computer and works without an internet connection. Follow the three steps below in order.

Step 1: Install a C Compiler

The compiler turns your `.c` file into a program you can run. Follow the instructions for your operating system.

Linux (Ubuntu or Debian)

1. Open a terminal (search for “Terminal” in your applications).
2. Type this command exactly and press Enter:

```
sudo apt update && sudo apt install build-essential
```

You will be asked for your password. Type it (nothing appears as you type; that is normal) and press Enter.

3. When it finishes, verify the installation:

```
gcc --version
```

You should see something like `gcc (Ubuntu 11.4.0) 11.4.0`. If you do, the compiler is ready.

macOS

1. Open a terminal (search for “Terminal” in Spotlight with `⌘ + Space`).
2. Type this command and press Enter:

```
xcode-select --install
```

A pop-up window appears asking you to install the command-line tools. Click **Install** and wait (this may take a few minutes).

3. Verify:

```
gcc --version
```

You should see output mentioning `Apple clang`. That is fine; it behaves the same as `gcc` for this course.

Windows

Windows does not come with a C compiler. You have two options:

Option W-1: WSL 2 (Recommended for Windows)

WSL 2 runs a real Linux environment inside Windows. It is the closest thing to a Linux terminal.

1. Open **PowerShell** as Administrator: press the Windows key, type `powershell`, right-click, and choose "Run as Administrator".
2. Type this command and press Enter:

```
wsl --install
```

3. Restart your computer when prompted.
4. After restart, open **Ubuntu** from the Start menu. The first time it runs, it will ask you to create a username and password.
5. In the Ubuntu terminal, follow the **Linux** steps above to install `gcc`.

Option W-2: MinGW-w64 (native Windows, no Linux layer)

1. Go to <https://winlibs.com/> and download the latest GCC release for Windows (look for a `.zip` file for your architecture, usually 64-bit).
2. Extract the archive. Move the extracted folder (e.g., `mingw64`) to `C:\mingw64`.
3. Add the compiler to your PATH:
 - Press Windows + R, type `sysdm.cpl`, press Enter.
 - Click **Advanced** → **Environment Variables**.
 - Under "System variables", find `Path`, click **Edit**, then **New**, and type `C:\mingw64\bin`. Click OK.
4. Open a new Command Prompt and verify:

```
gcc --version
```

In plain words: PATH PATH is a list of folders the computer searches through when you type a command. When you type `gcc`, the terminal looks through each folder in PATH and runs the first `gcc` it finds. Adding `C:\mingw64\bin` to PATH tells Windows where to find the compiler. Without this, the terminal would say '`gcc`' is not recognized as a command.

Step 2: Install VS Code (Recommended Editor)

Visual Studio Code (VS Code) is a free, lightweight code editor from Microsoft. It works on Windows, macOS, and Linux. For this course it is the recommended choice because it has a built-in terminal, instant error highlighting, and a free C extension.

Why VS Code? Think of VS Code as a specialised workshop for writing code. It has a file browser on the left, an editor in the centre, and a terminal at the bottom. You can write, compile, and run your program without leaving the window.

Install VS Code: step by step:

1. Go to <https://code.visualstudio.com/> and click the large **Download** button. Download the installer for your operating system.
2. Run the installer and follow the prompts. Accept the licence, choose the default install location, and click **Install**.
 - On Windows: check the boxes “Add to PATH” and “Add ‘Open with Code’ to context menu” when offered.
3. Open VS Code after the installer finishes.

Install the C/C++ Extension Pack:

The base VS Code cannot compile C. The extension pack adds syntax highlighting, error squiggles, and awareness of C-specific keywords.

1. In VS Code, click the **Extensions** icon in the left sidebar (it looks like four squares).
2. In the search box that appears, type: `C/C++ Extension Pack`.
3. Click the result published by **Microsoft** and click **Install**.
4. Wait for the installation to complete (a few seconds to a minute).

Step 3: Your First Program in VS Code

Follow these steps to write, compile, and run `hello.c` from inside VS Code.

Create your working folder:

1. On your computer, create a folder called `cprog-labs` somewhere easy to find (for example, inside your Documents folder).
2. In VS Code, go to **File → Open Folder**, navigate to `cprog-labs`, and click **Select Folder**.

Write `hello.c`:

3. In VS Code's Explorer panel (left sidebar), click the **New File** icon and name the file `hello.c`.
4. Type this code into the file exactly as written:

```
#include <stdio.h>

int main(void) {
    printf("Hello, world!\n");
    return 0;
}
```

5. Save the file: **Ctrl + S** (Windows/Linux) or **Cmd + S** (macOS).

Open the integrated terminal:

6. Go to **Terminal → New Terminal** (or press `Ctrl + ``). A terminal pane opens at the bottom of the VS Code window. You should see a prompt showing your `cprog-labs` folder path.

Compile:

7. In the terminal, type this command and press Enter:

```
gcc hello.c -o hello -Wall
```

If there are no errors, the terminal shows a new prompt with no output. That is correct. Silence means success at compile time.

Run:

8. Type the run command for your operating system and press Enter:

<code>./hello</code>	# Linux, macOS, or WSL 2
<code>hello.exe</code>	# Windows MinGW (native)

Expected output:

```
Hello, world!
```

If you see that line, your toolchain is fully working.

Why `./hello` and not just `hello`? The `.` means “the current folder”. Without it, the terminal would search through PATH to find a program called `hello`, not find one, and give an error. Writing `./hello` tells the terminal exactly where to look.

Alternatives to VS Code

Editor	Best for	Notes
Code::Blocks	Windows beginners who want one installer	Bundles a compiler and editor together. Download from https://www.codeblocks.org/ ; choose the version labelled “mingw-setup.exe” to get a compiler included. No separate PATH setup needed.
OnlineGDB / Programiz	Fallback when nothing is installed	Already covered in Option A above.
Vim / Neovim	Experienced users who prefer keyboard-only	Powerful, but the learning curve is steep for a complete beginner.

Compiler Flags We Use in This Course

```
gcc file.c -o file -Wall -Wextra -std=c99
```

Flag	Meaning
<code>-Wall</code>	Enable common warnings
<code>-Wextra</code>	Enable extra warnings
<code>-std=c99</code>	Use C99 standard (what this course targets)

Always compile with `-Wall -Wextra`. Warnings are not errors, but they almost always point to a real problem. Fix every warning before moving on.

Verify Checklist

Before you come to Lab 01, confirm all four items:

- ☐ `gcc --version` prints a version number in the terminal
- ☐ `hello.c` compiles with `gcc hello.c -o hello -Wall` and shows no errors or warnings
- ☐ Running `./hello` (or `hello.exe`) prints `Hello, world!`
- ☐ VS Code opens `.c` files and shows the code in different colours (syntax highlighting)

Once all four boxes are checked, you are ready for Lab 01.

How a Program Runs

This page gives you a **mental model**: a short, accurate picture of what happens between typing your code and seeing output. You do not need to memorize this; you need to *believe it*, because it explains many things that would otherwise seem like magic.

The Big Picture: Write, Compile, Run

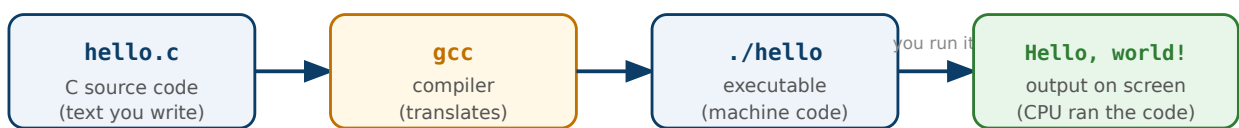


Figure S.1. From source code to a running program.

That is it. Three stages: **write, compile, run**.

Think of it like a recipe: you write the recipe (source code), a translator reads it and turns it into precise kitchen instructions any chef can follow (machine code), and then the chef (CPU) follows those instructions and produces the dish (output).

Stage 1: Write

You write **C source code** in a text file (`hello.c`). This is just text; the computer cannot run it yet. C is called a *high-level* language because it is close to how humans think, not how the CPU thinks.

Stage 2: Compile

When you run `gcc hello.c -o hello`, the compiler does four things in order:

1. **Reads and checks your text** for grammar mistakes (missing `;`, typos in keywords, etc.).
2. **Translates it into machine code**: the tiny numeric instructions the CPU understands. Machine code looks like `01001000 10001001 11000001 ...` (not human-readable, but exactly what the chip needs).

3. **Links in standard library code.** When you write `printf(...)`, the compiler does not invent `printf` from scratch; it copies the ready-made `printf` code from the standard library into your executable.
4. **Writes the executable file** (`hello`). This is the file you distribute or run.

In plain words: standard library The C standard library is a collection of pre-written, tested functions that come with every C compiler. `printf`, `scanf`, `strlen`, `sqrt` are all in it. You tell the compiler which part you need with `#include <stdio.h>` (for input/output functions) or `#include <math.h>` (for math functions). Think of the library as a toolbox that comes pre-packed with the compiler; `#include` picks up the tools you need for that project.

If there is a syntax error, the compiler prints an error message and does *not* produce an executable. **This is good.** It means the bug is caught before the program ever runs.

Stage 3: Run

When you type `./hello`, the operating system:

1. Loads the executable into **RAM (main memory)**.
2. Hands the first instruction to the **CPU**.
3. The CPU runs instructions one at a time, mostly top to bottom.

CPU and Memory: The Hardware Picture

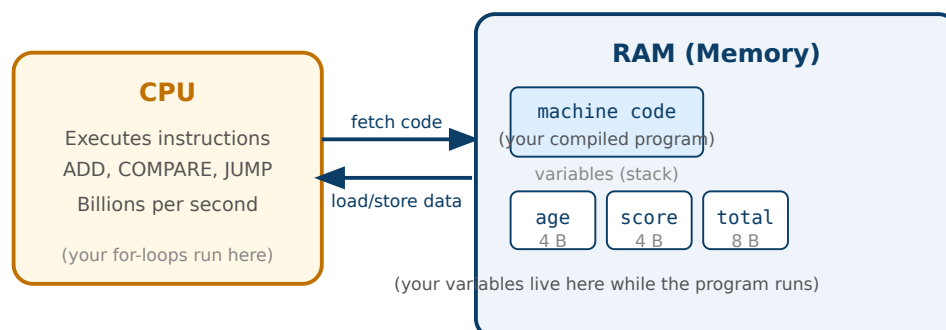


Figure S.2. How the CPU and RAM work together.

Think of RAM as a giant wall of numbered post-it notes. Each note holds a tiny piece of data. When your program declares a variable, the OS hands it a note (or several adjacent notes) and

writes the value there. The CPU reads and writes those notes at billions of operations per second. When the program finishes, all the notes are erased.

Variables live in RAM while your program runs. When the program ends, their memory is freed.

Under the Hood: what the CPU actually does

Your CPU is just a chip that can do a handful of things very fast: load a value from memory, store a value to memory, add two numbers, compare two numbers, and jump to a different instruction. Everything your program does (`printf`, `if`, `for`) is compiled down to combinations of these primitive operations.

This is why learning C is useful: you are close enough to the hardware to see the machine, but far enough away to think in human terms. In **Computer Architecture** you will study the CPU itself. In **System Programming** you will control memory and processes directly.

What Happens on a Runtime Error?

Some bugs are not caught by the compiler; they only appear when the program runs. For example, dividing by zero, or writing past the end of an array. These cause the program to crash, often with a message like `Segmentation fault`.

This is the second kind of bug you will learn to find and fix in this course.

Key Vocabulary

Term	Meaning
Source code	The C text you write (<code>.c</code> files)
Compiler	The tool (<code>gcc</code>) that translates source to machine code
Executable / binary	The machine-code file you run (<code>./hello</code>)
RAM	Memory where variables and code live while the program runs
Runtime error	A bug that crashes the running program, not caught at compile time
Syntax error	A grammar mistake the compiler catches before running

Lab 01: Course Introduction & Programming Concepts

Week	1
Duration	3 × 50 min (150 min)
Method	Lecture (no code submission this week)
Prerequisites	None

Why this lab matters: Every digital device runs on programs: your phone, your car's navigation system, your bank's ATM. This lab shows you where programs come from and how a human idea becomes instructions a computer can follow. Understanding the programming mindset of breaking a problem into small, ordered steps is a skill you will apply in every role that involves computing, whether you end up writing software, analyzing data, or managing systems.

Time allocation

Period	Min	Activity
A	50	30 min course overview and policies · 20 min programming concepts (problem to algorithm to code, pseudocode)
B	50	30 min environment setup walkthrough (compiler/editor and online-compiler fallback) · 20 min start hands-on
C	50	40 min hands-on: everyone types and runs a provided program · 10 min wrap-up and Week 2 preview

Learning Outcomes

By the end of this lab, you will be able to:

1. State what a program is and describe the problem-to-algorithm-to-code pipeline.
2. Write a simple pseudocode or flowchart for an everyday problem.
3. Open your text editor, create a `.c` file, and run it with a compiler (or online).
4. Locate and describe the course assessment model (Midterm 20%, Project 30%, Final 30%, Attendance 10%, Assignments 10%).

Recap

This is the first meeting; there is no prior week to recap.

Background

What Is Programming?

A **program** is a set of instructions that tells a computer what to do. You, the programmer, write those instructions in a **programming language** (C in this course). The computer does exactly what you wrote: nothing more, nothing less.

In plain words: programming language A programming language is a precise, structured way of writing instructions that both a human can read and a compiler can translate into machine code. English is too ambiguous for a computer (what does “make it bigger” mean exactly?). C uses strict grammar rules so there is only one possible interpretation of every instruction.

Think of it like the difference between telling a friend “make the text bigger” and writing in a legal contract “increase the font size to 14 points in Helvetica Bold.” The computer needs the contract version.

In plain words: algorithm An algorithm is a step-by-step plan for solving a problem. Every step must be clear and unambiguous. An algorithm is language-independent: the tea-making steps below could be translated into C, Python, or carried out by a robot.

From Problem to Code

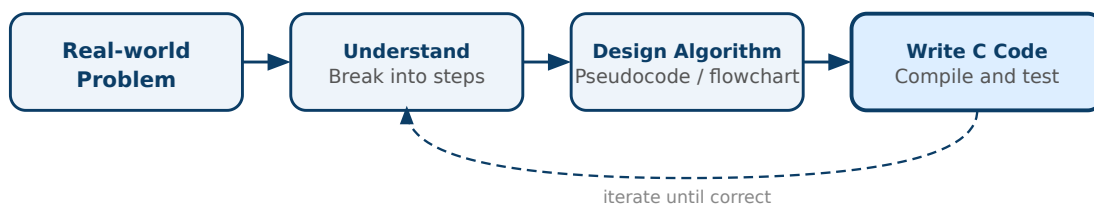


Figure 1.1. From problem to working code.

Before writing any code, you should be able to describe your algorithm in plain language or pseudocode.

What Is an Algorithm?

An algorithm is a precise, finite sequence of steps that solves a problem.

Example: make tea

1. Fill kettle with water.
2. Boil water.
3. Place tea bag in cup.
4. Pour boiling water into cup.
5. Wait 3 minutes.
6. Remove tea bag.
7. Add milk and sugar if desired.

Good programs come from good algorithms, not the other way around.

Pseudocode

Pseudocode is informal, algorithm-level writing that looks like code but is not tied to any language. There is no single correct pseudocode syntax; clarity is what matters. Its purpose is to think through the logic before worrying about C syntax. You cannot run pseudocode through a compiler; it is a planning tool, like a rough sketch before a final drawing.

Example: find the larger of two numbers

```
INPUT a, b
IF a > b THEN
    OUTPUT a
ELSE
    OUTPUT b
END IF
```

Under the Hood: what is a computer?

At its core, a computer is a machine that can store data (in RAM) and run instructions (in the CPU). The CPU performs a few billion simple operations per second: add, subtract, compare, jump, load, store. Everything a program does (printing text, sorting a list, playing a video) is built from combinations of these primitives. In this course you will write programs in C, which is close enough to the machine that you will start to see this.

Worked Examples

Example 1: The provided “first program” (run, do not write yet)

This program will be provided as a file. Read it, then run it:

```
/* first.c */
#include <stdio.h>

int main(void) {
    printf("Hello, Computer Programming I!\n");
    printf("Instructor: Yushintia Pramitarini\n");
    return 0;
}
```

Expected output:

```
Hello, Computer Programming I!
Instructor: Yushintia Pramitarini
```

You do not need to understand every line yet; that is Week 2's job. The goal right now is: does it compile and run on *your* machine?

Guided In-Lab Exercises

Exercise 1: Verify your toolchain (Period B)

- Follow [Setup: Toolchain](#) to install gcc and an editor.
- Alternatively, open OnlineGDB or Programiz in a browser.
- Type (do not copy-paste) the example above, compile, and run it.
- Raise your hand if you see an error you cannot resolve.

Exercise 2: Pseudocode for a vending machine (Period B and C)

Write pseudocode (not code) for the following problem:

A vending machine sells drinks for 1,500 won each. A customer inserts coins. The machine should dispense a drink and return change when the customer has inserted enough coins.

Your pseudocode should handle:

- Accumulating the total inserted so far
- Checking when the total is at least 1,500
- Computing and dispensing change

There is no single correct answer. Discuss your solution with the person next to you.

Exercise 3: Run the provided program (Period C)

- Open `first.c` (provided by the instructor).
 - Compile it: `gcc first.c -o first`
 - Run it: `./first`
 - Take a screenshot of the terminal showing the output.
-

Challenge Problem

Design a flowchart (on paper or a drawing tool) for the following:

A student wants a program that reads their scores for three exams and prints their average, and whether they passed (average at least 60) or failed.

You will implement this as actual C code in Week 5.

Common Pitfalls

- **"I cannot find the terminal."** On Windows: search for "Command Prompt", "PowerShell", or "WSL". On macOS: search for "Terminal". On Linux: right-click the desktop and look for Terminal.
 - **"The compiler says 'command not found'."** gcc is not installed or not on your PATH. Use the online compiler today and fix the install before Week 2.
 - **Pseudocode is not code.** Do not worry about semicolons or exact syntax. Write it so a classmate could follow the steps.
-

Submission and Rubric

Deliverable	Filename	Points
Screenshot of <code>first.c</code> running on your machine (or online compiler)	<code>lab01_screenshot.png</code> (or <code>.jpg</code>)	5
Pseudocode for the vending machine problem (handwritten or typed)	<code>lab01_pseudo.txt</code> (or photo)	5

Total: 10 points (graded per [Assignments rubric](#))

No code to compile this week; the screenshot proves your environment works.

Further Reading

- K. N. King, *C Programming: A Modern Approach*, Ch. 1 “Introducing C”
- [Setup: Toolchain](#)
- [Setup: How a Program Runs](#)

Lab 02: Program Structure & Basic I/O

Week	2
Duration	3 × 50 min (150 min)
Method	Lecture & Lab
Prerequisites	Lab 01 (working toolchain)

Why this lab matters: A program that cannot receive input or display output is like a calculator with no buttons and no screen. This lab gives your programs the ability to communicate: to ask the user for data and to report results back. Every interactive application you have ever used, from a login screen to a search box to a form on a website, depends on exactly these operations, which makes this lab the foundation for everything you will build in this course.

Time allocation

Period	Min	Activity
A (Concept)	50	5 recap · 30 program anatomy and compilation model · 15 live demo
B (Guided practice)	50	40 guided <code>printf</code> coding and reading errors · 10 debrief and pitfalls
C (Independent and wrap)	50	35 independent exercises · 10 challenge · 5 submit and Week 3 preview

Learning Outcomes

By the end of this lab, you will be able to:

1. Identify the parts of a minimal C program: preprocessor directives, `main`, statements, `return`.
2. Use `printf` to print text, numbers, and escape sequences (`\n`, `\t`).
3. Compile a program with `gcc` and read the first error message when it fails.
4. Deliberately introduce a common error, read the message, and fix it.

Recap

In Week 1 you ran a program someone else wrote and verified that your toolchain works. This week you write your first programs from scratch and learn what each part does.

Background

Anatomy of a C Program

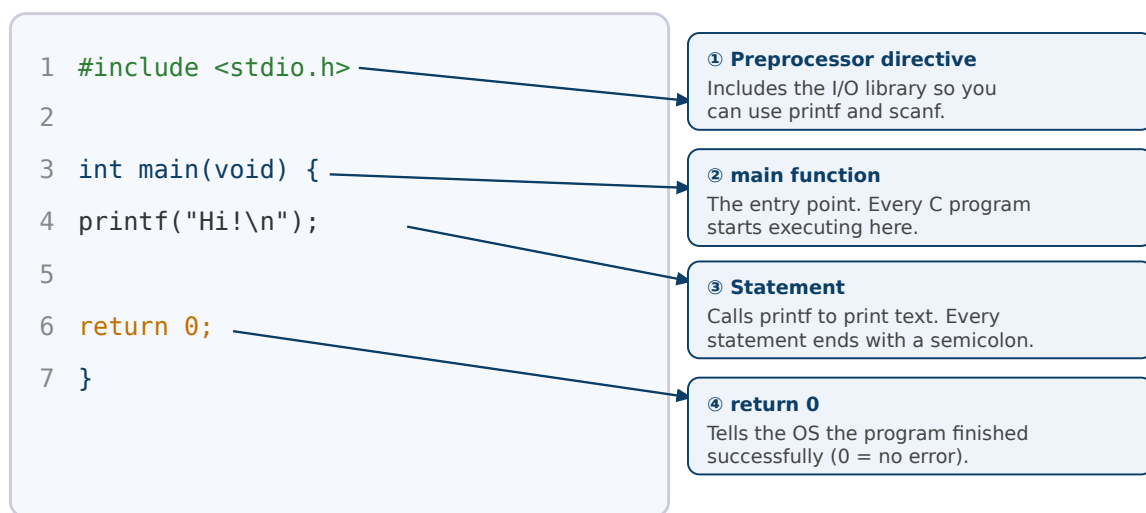


Figure 2.1. Anatomy of a C program.

Part	What it does
<code>#include <stdio.h></code>	Includes the standard input/output library so you can use <code>printf</code> and <code>scanf</code> .
<code>int main(void)</code>	The entry point. Every C program starts here. <code>int</code> means <code>main</code> returns an integer; <code>void</code> means it takes no arguments.
<code>{ ... }</code>	Curly braces group the body of <code>main</code> . Everything between <code>{</code> and <code>}</code> belongs to <code>main</code> .
<code>printf("Hi!\n");</code>	Calls <code>printf</code> to print text. <code>\n</code> is a newline (moves to the next line). Every statement ends with <code>;</code> .
<code>return 0;</code>	Tells the operating system the program finished successfully. 0 means "no error"; any other number signals a problem.

In plain words: library A library is a collection of pre-written, pre-tested functions that come with the compiler. Instead of writing your own code to talk to the screen, you use `printf`, which was written by experts and works on every platform. `#include <stdio.h>` is the instruction that brings the input/output library (standard I/O) into your program. Think of it like borrowing a professional toolkit from a shelf rather than building every tool yourself.

In plain words: main function Every C program must have exactly one function named `main`. When you run the program, the operating system starts executing from the first line inside `main`. It is the front door of your program. There can only be one front door.

In plain words: statement and semicolon A statement is one complete instruction, like one sentence in English. In C, every statement ends with a semicolon (`;`). The semicolon is the full stop. Forgetting it is the single most common compile error you will make, and the compiler will tell you exactly where the missing one is.

Compile and Run: in One Sentence

Your code goes into `gcc`, which produces a program you run.

```
gcc hello.c -o hello -Wall    # compile
./hello                      # run
```

printf and Escape Sequences

```
printf("Line one\n");        // \n means newline
printf("A\tB\tC\n");         // \t means tab
printf("Quote: \"Hi\"\n");    // \" means a literal double-quote
```

Reading Error Messages

When `gcc` refuses to compile, it prints something like:

```
hello.c:5:5: error: expected ';' before 'return'
```

Read it as: **file** `hello.c`, **line 5, column 5: missing semicolon before** `return`.

Rules:

1. Find the **line number** and look at that line first.
2. Fix the **first** error; recompile. Later errors often disappear.
3. Read the message literally; it usually tells you exactly what is wrong.

Under the Hood: compilation

When you run `gcc`, it translates your human-readable C text into machine code: numbers the CPU can execute directly. The resulting executable file is entirely different from your source. If your source has a syntax error, the compiler stops and produces nothing. It cannot translate something it does not understand.

Worked Examples

Example 1: Hello, World with formatting

```
/* lab02_hello.c */
#include <stdio.h>

int main(void) {
    printf("=====\n");
    printf("  Computer Programming I  \n");
    printf("  Instructor: Y. Pramitar  \n");
    printf("=====\n");
    return 0;
}
```

Expected output:

```
=====
  Computer Programming I
  Instructor: Y. Pramitar
=====
```

Example 2: Printing numbers and mixing text

```
/* lab02_numbers.c */
#include <stdio.h>

int main(void) {
    printf("My favourite number is %d\n", 42);
    printf("Pi is approximately %.2f\n", 3.14159);
    printf("The letter A has ASCII code %d\n", 'A');
    return 0;
}
```

Expected output:

```
My favourite number is 42
Pi is approximately 3.14
The letter A has ASCII code 65
```

%d is a *format specifier*: a placeholder that tells `printf` what type of value to insert and how to display it. **%d** means “insert an integer here in decimal.” **%.2f** means “insert a decimal number here, showing 2 digits after the decimal point.” You will use these every week; Week 4 covers all the options in detail.

Guided In-Lab Exercises

Exercise 1: Your name card (Period B)

Write a program that prints a name card in this format:

```
+-----+
| Name : [your name]      |
| ID   : [your student ID] |
| Lab  : Computer Prog. I  |
+-----+
```

File: `lab02_card.c`

Exercise 2: Break it and read the error (Period B)

Take your `lab02_card.c` and:

1. Remove the semicolon from one line. Compile. Read the error. Put it back.
2. Remove the `#include <stdio.h>` line. Compile. Read the error. Put it back.
3. Change `main` to `Main`. Compile. Read the error. Put it back.

For each case, write a comment at the top of your file recording what the error message said.

Exercise 3: ASCII banner (Period C)

Write a program that prints a simple ASCII banner of your choice (for example, your initials drawn with `*` characters) using `printf`. Use at least 5 `printf` calls and at least one escape sequence other than `\n`.

Challenge Problem

Write a program that prints a multiplication table header for the 5-times table:

```
5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50
```

Write out all 10 `printf` calls by hand (we will use loops starting Week 6).

Common Pitfalls

Mistake	Error message (approx.)	Fix
Missing <code>;</code> at end of statement	<code>error: expected ';'</code>	Add the semicolon
Missing <code>#include <stdio.h></code>	<code>warning: implicit declaration of 'printf' or link error</code>	Add the include
Using <code>'</code> instead of <code>"</code> around strings	<code>error: expected expression</code>	Use double quotes for strings
Unclosed <code>{</code> or <code>}</code>	<code>error: expected declaration at end of file</code>	Count your braces

Submission and Rubric

Deliverable	Filename	Points
Name card program	lab02_card.c	5
ASCII banner	lab02_banner.c	5

Total: 10 points (graded per [Assignments rubric](#))

Further Reading

- King, Ch. 2 “C Fundamentals”
- [Debugging Tips](#)

Lab 03: Variables, Data Types & Expressions

Week	3
Duration	3 × 50 min (150 min)
Method	Lecture & Lab
Prerequisites	Lab 02 (printf, basic structure)

Why this lab matters: Computers store everything, from a name to a price to a sensor reading to a pixel color, as numbers in memory. Choosing the right data type ensures your program stores values correctly, uses memory efficiently, and avoids errors such as storing a decimal as an integer and silently losing the fractional part. Data types are the vocabulary of programming: knowing them lets you express any real-world value precisely in code.

Time allocation

Period	Min	Activity
A (Concept)	50	5 recap · 30 types, declaration, expressions · 15 live demo
B (Guided practice)	50	40 guided <code>scanf</code> and interactive program · 10 debrief and pitfalls
C (Independent and wrap)	50	35 independent exercises (conversions) · 10 challenge · 5 submit and Week 4 preview

Learning Outcomes

By the end of this lab, you will be able to:

1. Declare variables of type `int`, `float` / `double`, and `char`.
2. Assign values, use constants (`const`, `#define`), and perform arithmetic.
3. Use `scanf` to read a value from the user, making programs interactive.
4. Explain that a variable is a named location in memory with a fixed type and size.

Recap

In Week 2 you learned the structure of a C program and used `printf` for output. Programs that only print fixed text are not very useful. This week you add **variables** (places to store

data) and `scanf` (a way to read data from the keyboard) so your programs can react to whatever the user types.

Background

What Is a Variable?

In plain words: variable A variable is a labelled storage box inside the computer's memory (RAM). You give it a name, say what kind of data it holds (its *type*), and the computer reserves the right amount of space. Whenever you use the name in your code, the computer goes to that box and reads or writes the value inside.

Think of RAM as a long shelf of identical small boxes. Each box holds a tiny piece of information. A variable is one of those boxes with a sticky label on it so you can find it by name instead of by shelf number.

Variables as Labeled Boxes in Memory

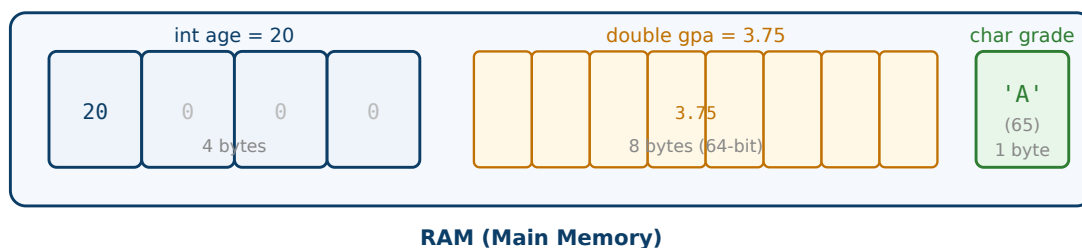


Figure 3.1. Variables as labeled boxes in RAM.

The diagram above shows how three variables look in RAM:

- `int age = 20` takes up **4 boxes** (4 bytes) on the shelf. The number 20 is stored in the first byte; the rest hold 0.
- `double gpa = 3.75` takes up **8 boxes** (8 bytes). Decimal numbers need more space to store the fractional part.
- `char grade = 'A'` takes up **1 box** (1 byte). The letter 'A' is stored as the number 65.

Each type uses a fixed amount of space. That is why you must tell C what type a variable is: the compiler needs to know how many bytes to reserve.

Data Types

In plain words: data type A data type tells the computer two things: how many bytes to reserve for a variable, and how to interpret the bits stored there. The same four bytes mean completely different things if you tell the computer “this is an integer” versus “this is a decimal number”. Choosing the right type means choosing the right-sized, right-shaped box.

Type	Size	Range / precision	Use
<code>int</code>	4 bytes	roughly -2 billion to +2 billion	whole numbers
<code>float</code>	4 bytes	~6 to 7 significant digits	decimal numbers (less precise)
<code>double</code>	8 bytes	~15 to 16 significant digits	decimal numbers (preferred)
<code>char</code>	1 byte	single character (e.g., 'A')	letters, symbols

Prefer `double` over `float` for decimal numbers in this course. `float` has less precision and causes hard-to-find rounding bugs.

Declaration and Initialization

In plain words: declaration vs initialization *Declaring* a variable means telling C “I need a box called X of type Y.” The box exists but its contents are unknown garbage until you put something in. *Initializing* means putting a starting value into the box at the same time you create it. Always initialize variables before reading them; reading an uninitialized box is a bug.

```
int age;           /* declaration only: box exists, value is garbage */
age = 20;          /* assignment: put 20 into the box */
int score = 95;    /* declaration AND initialization in one line */
```

Think of it like buying a new notebook. Declaring it is buying the notebook (it exists). Initializing it is writing your name on the first page right away.

Constants

Two ways to create a value that should not change:

In plain words: `const` and `#define` `const double PI = 3.14159;` creates a named constant. The compiler knows the type and warns you if you try to change it. This is the safest way.

`#define MAX 100` is a preprocessor shortcut. Before compiling, the preprocessor finds every word `MAX` in your file and replaces it with `100`. It does not know about types. Think of it like a find-and-replace in a text editor that runs before the compiler even starts. Use `const` for most constants; use `#define` when you need a simple integer or string label shared across multiple files.

```
const double PI = 3.14159; /* typed constant: the compiler will not let you change
it */
#define MAX 100             /* preprocessor replacement: MAX becomes 100 everywhere
*/
```

Expressions and Arithmetic

```
int a = 10, b = 3;
int sum = a + b; /* 13 */
int diff = a - b; /* 7 */
int prod = a * b; /* 30 */
int quot = a / b; /* 3 (integer division: decimal part is dropped) */
int rem = a % b; /* 1 (remainder after division: 10 = 3x3 + 1) */
```

Integer division: `10 / 3` gives `3`, not `3.333...`. The decimal part is simply dropped, not rounded. This is called **truncation**. To get the decimal result, force one of the numbers to be a decimal by casting: `(double)a / b` gives `3.333...`

In plain words: truncation vs rounding Truncation means “cut off the decimal part, no matter what.” So 3.9 becomes 3, and 3.1 also becomes 3. It is NOT rounding. Rounding would turn 3.9 into 4, but truncation always goes toward zero. Integer division in C always truncates.

Reading Input with scanf

Up to now your programs printed fixed text. `scanf` makes them interactive: they ask the user a question and use whatever the user types.

```
int x;
printf("Enter a number: ");
scanf("%d", &x);
printf("You entered: %d\n", x);
```

In plain words: the `&` in `scanf` `&x` means “the memory address of the box called `x`”; in other words, where on the RAM shelf the box `x` is located. `scanf` needs this address so it can write the value the user types directly into the right box. Without `&`, you are telling `scanf` the *contents* of the box (the current garbage value), not *where* the box is. This is one of the most common beginner mistakes and causes a program crash. Always write `scanf("%d", &x)`, not `scanf("%d", x)`.

You will understand addresses more deeply in Week 13 (pointers). For now, just remember: `scanf` always needs `&` in front of each variable name.

Format specifier	Use with
<code>%d</code>	<code>int</code>
<code>%f</code>	<code>float</code>
<code>%lf</code>	<code>double</code>
<code>%c</code>	<code>char</code>

Under the Hood: a variable is a labeled box of bytes

When you declare `int age;`, the computer sets aside 4 bytes of RAM and gives them the label `age`. The type `int` tells the compiler how many bytes to use and how to interpret the bits stored there. This is why `int / int` drops the decimal: the result box can only hold an integer, so any fractional part is thrown away when the result is stored.

Worked Examples

Example 1: Interactive greeting

```
/* lab03_greet.c */
#include <stdio.h>

int main(void) {
    int age;
    char initial;
    printf("Enter your first initial: ");
    scanf(" %c", &initial);
    printf("Enter your age: ");
    scanf("%d", &age);
    printf("Hello, %c! You are %d years old.\n", initial, age);
    return 0;
}
```

Line by line

Line	What it does	Why
<code>#include <stdio.h></code>	Pulls in the I/O library	Gives us <code>printf</code> and <code>scanf</code>
<code>int age;</code>	Declares a box called <code>age</code> that holds a whole number	We will store the user's age here
<code>char initial;</code>	Declares a box called <code>initial</code> that holds one character	We will store the user's first letter here
<code>printf("Enter your first initial: ");</code>	Prints a prompt on screen (no newline)	Tells the user what to type next
<code>scanf(" %c", &initial);</code>	Waits for the user to type a character; stores it in <code>initial</code>	The space before <code>%c</code> skips any leftover whitespace (like a newline from a previous Enter press)
<code>printf("Enter your age: ");</code>	Prints the second prompt	Tells the user what to type next
<code>scanf("%d", &age);</code>	Waits for the user to type an integer; stores it in <code>age</code>	<code>%d</code> matches an <code>int</code> ; <code>&age</code> is the address of the box
<code>printf("Hello, %c! You are %d years old.\n", initial, age);</code>	Prints the result using both stored values	<code>%c</code> is replaced by the letter; <code>%d</code> is replaced by the number
<code>return 0;</code>	Tells the OS the program finished without errors	A return value of 0 means success

Sample run:

```
Enter your first initial: A
Enter your age: 20
Hello, A! You are 20 years old.
```

Example 2: Integer vs float division

```
/* lab03_division.c */
#include <stdio.h>

int main(void) {
    int a = 7, b = 2;
    printf("int / int = %d\n", a / b);           /* 3 */
    printf("cast to double = %.2f\n", (double)a / b); /* 3.50 */
    return 0;
}
```

`(double)a` converts the value of `a` to a decimal number before the division happens. When C divides a decimal by an integer, it promotes the integer to decimal first, so the result is `3.50` instead of `3`.

Guided In-Lab Exercises

Exercise 1: Temperature converter (Period B)

Write a program that:

1. Asks the user for a temperature in Celsius.
2. Converts it to Fahrenheit: $F = C \times \frac{9}{5} + 32$
3. Prints the result with 2 decimal places.

File: `lab03_temp.c`

Watch out: `9/5` is integer division (result: 1). Use `9.0/5.0` or cast.

Exercise 2: Circle geometry (Period B and C)

Write a program that:

1. Reads the radius from the user.
2. Computes area = $\pi \times r^2$ and circumference = $2 \times \pi \times r$.
3. Prints both with 4 decimal places.

Use `const double PI = 3.14159265;`

File: `lab03_circle.c`

Exercise 3: Swap two values (Period C)

Ask the user for two integers. Swap their values and print them after the swap. Hint: you need a temporary variable (a third box to hold one value while you rearrange the other two).

File: `lab03_swap.c`

Challenge Problem

Write a program that reads a four-digit integer (e.g., 2025) and prints its individual digits:

```
Thousands: 2
Hundreds:  0
Tens:      2
Units:     5
```

Hint: use integer division and modulo. For example, `2025 / 1000` gives `2`, and `2025 % 1000` gives `025`.

Common Pitfalls

Mistake	Symptom	Fix
Forgetting <code>&</code> in <code>scanf</code>	Program crashes or reads garbage	Always use <code>&variable</code> with <code>scanf</code>
<code>int / int</code> when you want decimals	Result is truncated (e.g., <code>7/2</code> gives <code>3</code>)	Cast: <code>(double)a / b</code>
Wrong format specifier (<code>%d</code> for <code>double</code>)	Garbage output	Use <code>%lf</code> for <code>double</code> in <code>scanf</code> ; <code>%f</code> or <code>%lf</code> both work in <code>printf</code>
Reading an uninitialized variable	Unpredictable value (could be anything)	Always assign a value before reading
Using <code>=</code> where you need <code>==</code>	(More relevant in Week 5, but start noticing now)	<code>=</code> assigns; <code>==</code> compares

Submission and Rubric

Deliverable	Filename	Points
Temperature converter	lab03_temp.c	4
Circle geometry	lab03_circle.c	4
Swap	lab03_swap.c	2

Total: 10 points

Further Reading

- King, Ch. 3 “Formatted Input/Output”, Ch. 4 “Expressions”
- King, Ch. 7 “Basic Types”

Lab 04: Input/Output & Operators

Week	4
Duration	3 × 50 min (150 min)
Method	Lecture & Lab
Prerequisites	Lab 03 (variables, scanf)

Why this lab matters: Every program that calculates something, such as a tax bill, a body-mass index, a GPS distance, or a score in a game, uses arithmetic and logical expressions. This lab gives you the tools to build those calculations and to control exactly how values are formatted when displayed. The ability to write correct expressions is as fundamental to programming as arithmetic is to mathematics.

Time allocation

Period	Min	Activity
A (Concept)	50	5 recap · 30 formatted I/O, operators, and precedence · 15 live demo
B (Guided practice)	50	40 guided calculator lab · 10 debrief and pitfalls (int division, cast)
C (Independent and wrap)	50	35 independent exercises · 10 challenge · 5 submit and Week 5 preview

Learning Outcomes

By the end of this lab, you will be able to:

1. Format `printf` output using width, precision, and alignment specifiers.
2. Use arithmetic, relational, and logical operators correctly.
3. Predict the result of a mixed-type expression and apply explicit casts.
4. Explain operator precedence and use parentheses to override it.

Recap

In Week 3 you stored data in variables and read it with `scanf`. This week you learn to control *how* output looks (formatted I/O) and deepen your understanding of the operations you can

perform on data.

Background

Formatted Output: Width and Precision

In plain words: field width When you write `%8d`, the `8` is the *minimum field width*: `printf` will use at least 8 characters to print the number, padding with spaces on the left if the number is shorter. Think of it like a fixed-width column in a table. Every number lines up at the right edge of its column, no matter how many digits it has.

In plain words: precision When you write `%.4f`, the `.4` is the *precision*: how many digits to show after the decimal point. `printf("%.2f", 3.14159)` prints `3.14`; the extra digits are cut off (not rounded mathematically; the display is rounded, but the stored value is unchanged).

```
printf("%8d\n", 42);           /* right-aligned in 8-char field: "    42" */
printf("%-8d|\n", 42);         /* left-aligned:                "42    |" */
printf("%08d\n", 42);          /* zero-padded:                  "00000042" */
printf("%.4f\n", 3.14159);     /* 4 decimal places:            "3.1416" */
printf("%10.2f\n", 3.14);      /* width 10, 2 decimal places   "          3.14" */
```

Reading a format specifier: `%10.2f` breaks down as:

- `%` : start of a format specifier
- `10` : minimum field width (at least 10 characters)
- `.2` : precision (2 digits after the decimal point)
- `f` : type: floating-point number

Operator Precedence Pyramid

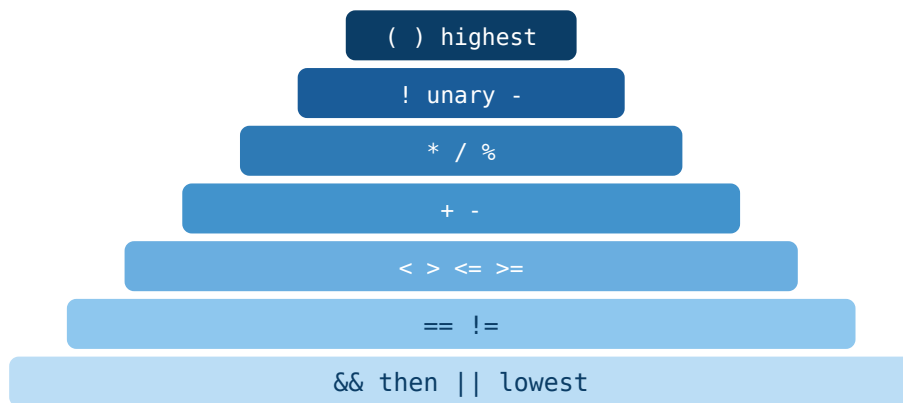


Figure 4.1. Operator precedence, highest to lowest.

How to read the pyramid: operators near the top are evaluated first (higher precedence). Operators near the bottom are evaluated last. So in the expression `2 + 3 * 4`, the `*` (level 3) happens before `+` (level 4), giving `2 + 12 = 14`, not `5 * 4 = 20`.

When two operators are at the same level, they are usually evaluated left to right: `10 - 3 - 2` = `(10 - 3) - 2 = 5`, not `10 - (3 - 2) = 9`.

When in doubt: **use parentheses**; they cost nothing and prevent bugs.

Arithmetic Operators

Op	Meaning	Example	Result
<code>+</code>	Add	<code>3 + 4</code>	<code>7</code>
<code>-</code>	Subtract	<code>7 - 2</code>	<code>5</code>
<code>*</code>	Multiply	<code>3 * 4</code>	<code>12</code>
<code>/</code>	Divide	<code>7 / 2</code>	<code>3 (int!)</code> , <code>3.5 (float)</code>
<code>%</code>	Modulo (remainder)	<code>7 % 2</code>	<code>1</code>

Relational and Logical Operators

Op	Meaning
<code>==</code> , <code>!=</code>	equal, not equal
<code><</code> , <code>></code> , <code><=</code> , <code>>=</code>	comparisons
<code>&&</code>	logical AND
<code> </code>	logical OR
<code>!</code>	logical NOT

These return `1` (true) or `0` (false). You will use them heavily in Week 5.

Type Conversion and Casts

In plain words: cast A cast is a way of telling the compiler: “treat this value as a different type, just for this one calculation.” Writing `(double)a` does not change what is stored in `a`; it makes a temporary copy of `a`’s value as a decimal number for the division. Think of it like asking someone to read a measurement in inches when it was originally written in centimeters: the paper does not change, but the interpretation does.

```
int a = 7, b = 2;
double result = (double)a / b; /* cast a to decimal; result is 3.5 */
int truncated = (int)3.99;     /* cast to int: 3 (truncation, NOT rounding) */
```

Remember: truncation always goes toward zero. `(int)3.99 = 3`. `(int)-3.99 = -3`.

Under the Hood: operators map to CPU instructions

Each arithmetic operator compiles down to a single CPU instruction (ADD, SUB, MUL, DIV). The CPU works in a specific number of bits (32 or 64), which is why `int / int` discards the remainder. The integer division instruction throws it away. A cast changes how the compiler interprets the bits.

Worked Examples

Example 1: Formatted table

```
/* lab04_table.c */
#include <stdio.h>

int main(void) {
    printf("%-10s %8s %10s\n", "Item", "Qty", "Price");
    printf("%-10s %8d %10.2f\n", "Pencil", 5, 1500.0);
    printf("%-10s %8d %10.2f\n", "Notebook", 2, 8500.0);
    printf("%-10s %8d %10.2f\n", "Eraser", 10, 500.0);
    return 0;
}
```

Expected output:

Item	Qty	Price
Pencil	5	1500.00
Notebook	2	8500.00
Eraser	10	500.00

Line by line

Let’s decode the header row: `printf("%-10s %8s %10s\n", "Item", "Qty", "Price");`

Format piece	Value	What it prints	Why
<code>%-10s</code>	<code>"Item"</code>	Item (left-aligned in 10 chars)	- means left-align; 10 is the column width; s means string
<code>%8s</code>	<code>"Qty"</code>	Qty (right-aligned in 8 chars)	No -, so right-align; 8-char column
<code>%10s</code>	<code>"Price"</code>	Price (right-aligned in 10 chars)	10-char column
<code>\n</code>	(none)	moves to the next line	newline character

Every data row uses the same column widths (`%-10s` , `%8d` , `%10.2f`), so the numbers line up vertically. This is why formatted output matters for tables and receipts.

Example 2: Integer division pitfall

```
/* lab04_divide.c */
#include <stdio.h>

int main(void) {
    int x = 5, y = 2;
    printf("5 / 2      = %d\n", x / y);          /* 2 */
    printf("5.0 / 2    = %.1f\n", 5.0 / y);      /* 2.5 */
    printf("(double)5/2 = %.1f\n", (double)x / y); /* 2.5 */
    printf("5 %% 2      = %d\n", x % y);          /* 1 */
    return 0;
}
```

Guided In-Lab Exercises

Exercise 1: Receipt formatter (Period B)

Write a program that reads a unit price and quantity, then prints a receipt:

```
Item price : 12500.00
Quantity   :      3
-----
Subtotal   : 37500.00
Tax (10%)  :  3750.00
Total      : 41250.00
```

Use `printf` width and precision to align the numbers.

File: `lab04_receipt.c`

Exercise 2: Boolean expressions (Period B and C)

Write a program that reads two integers and prints the result (0 or 1) of:

- `a > b`
- `a == b`
- `a > 0 && b > 0`
- `!(a == b)`

File: `lab04_bool.c`

Exercise 3: Precedence puzzle (Period C)

Without running the code, predict the output of:

```
int x = 2 + 3 * 4 - 1;
int y = (2 + 3) * (4 - 1);
int z = 10 / 3 + 10 % 3;
printf("%d %d %d\n", x, y, z);
```

Then write the program, run it, and check your prediction.

File: `lab04_precedence.c` (include your predicted values in a comment)

Challenge Problem

Write a program that reads a price in Korean won and prints the breakdown in:

- 50,000-won notes
- 10,000-won notes
- 5,000-won notes
- 1,000-won notes
- Remaining coins (won less than 1000)

Example: 127,300 won gives 2 x 50,000 + 2 x 10,000 + 1 x 5,000 + 2 x 1,000 + 300 won coins.

Common Pitfalls

Mistake	Symptom	Fix
<code>int / int</code> gives unexpected 0 or truncated result	e.g., <code>1/2</code> gives <code>0</code>	Cast one operand: <code>(double)a / b</code>
Confusing <code>=</code> (assign) with <code>==</code> (compare)	Silent wrong results	<code>a = 5</code> sets <code>a</code> ; <code>a == 5</code> tests <code>a</code>
Forgetting <code>%%</code> to print a literal <code>%</code> in <code>printf</code>	<code>%</code> disappears or causes a warning	Use <code>%%</code>
Wrong precedence in complex expression	Unexpected result	Add parentheses

Submission and Rubric

Deliverable	Filename	Points
Receipt formatter	lab04_receipt.c	5
Boolean expressions	lab04_bool.c	3
Precedence puzzle	lab04_precedence.c	2

Total: 10 points

Further Reading

- King, Ch. 3 “Formatted Input/Output”
- King, Ch. 4 “Expressions”
- [Bonus: Bitwise Operators](#)

Lab 05: Conditional Statements

Week	5
Duration	3 × 50 min (150 min)
Method	Lecture & Lab + Diagnostic Assessment
Prerequisites	Lab 04 (operators, relational/logical)

Why this lab matters: Real programs must make decisions: charge a different tax rate for different income levels, show an error screen when a login fails, or flag a sensor reading only when it exceeds a threshold. Conditional statements are the mechanism that lets your code react to data rather than process everything the same way every time. Without them, a program can do only one fixed thing; with them, it can handle the full range of situations your users will encounter.

Time allocation

Period	Min	Activity
A (Concept)	50	5 recap · 30 if/else and switch · 15 live demo
B (Guided practice and diagnostic)	50	30 guided lab · 10 diagnostic assessment · 10 debrief and pitfalls
C (Independent and wrap)	50	35 independent exercises · 10 challenge · 5 submit and Week 6 preview

Learning Outcomes

By the end of this lab, you will be able to:

1. Write `if`, `if-else`, and nested `if` statements to branch program flow.
2. Combine conditions with `&&`, `||`, and `!`.
3. Write a `switch-case` statement and know when to prefer it over `if-else`.
4. Identify the `=` vs `==` bug and explain why it is dangerous.

Recap

In Week 4 you learned relational (`<`, `>`, `==`) and logical (`&&`, `||`) operators, which produce true/false (1/0) values. This week you use those values to make decisions: the program takes

different paths depending on the data.

Background

What Are Conditional Statements?

In plain words: conditional execution / branching So far, every program you wrote ran every line, top to bottom, every time. A conditional statement lets the program ask a question and take a different path depending on the answer. If the answer is YES, do this block; if NO, do that block. This is called *branching*, like a fork in a road.

Think of a vending machine: it checks whether you inserted enough money (the condition). If yes, it dispenses the drink (one branch). If no, it asks for more money (the other branch). Without branching, it would dispense a drink no matter what.

if and if-else Flowchart

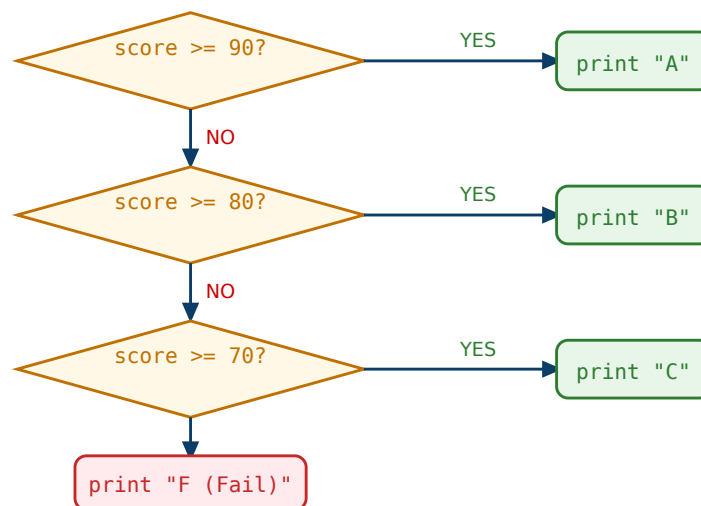


Figure 5.1. if / else-if decision flow.

```

if (score >= 90) {
    printf("A\n");
} else if (score >= 80) {
    printf("B\n");
} else if (score >= 70) {
    printf("C\n");
} else {
    printf("F\n");
}

```

switch-case

```

int day = 3;
switch (day) {
    case 1: printf("Monday\n"); break;
    case 2: printf("Tuesday\n"); break;
    case 3: printf("Wednesday\n"); break;
    default: printf("Unknown\n"); break;
}

```

- `switch` works on **integer** (and `char`) values only. It cannot test floating-point numbers or strings.
- `default` handles any value not matched by a `case`. Always include it.
- Always write `break` at the end of each case body.

In plain words: fall-through When a `switch` case matches, execution starts there and *keeps going* through the lines below until it hits a `break` or the end of the `switch`. This is called fall-through. If you forget `break`, the code “falls through” into the next case even though that case did not match. For example, if `day == 1` and case 1 has no `break`, then “Monday” AND “Tuesday” would both print. This is almost always a bug. There are rare intentional uses for fall-through, but in this course, always add `break`.

Under the Hood: branches as jumps

An `if` statement compiles to a **compare** instruction followed by a **conditional jump**. The CPU tests the condition; if it is false, execution jumps past the `if` body. Programs do not “know” which branch they will take; they decide at runtime. Multiple `else if` chains compile to a series of comparisons; `switch` with many cases often compiles to a **jump table** (faster).

Worked Examples

Example 1: Grade classifier

```
/* lab05_grade.c */
#include <stdio.h>

int main(void) {
    int score;
    printf("Enter score (0-100): ");
    scanf("%d", &score);

    if (score < 0 || score > 100) {
        printf("Invalid score.\n");
    } else if (score >= 90) {
        printf("Grade: A\n");
    } else if (score >= 80) {
        printf("Grade: B\n");
    } else if (score >= 70) {
        printf("Grade: C\n");
    } else if (score >= 60) {
        printf("Grade: D\n");
    } else {
        printf("Grade: F (Fail)\n");
    }
    return 0;
}
```

Line by line

Line	What it does
<code>int score;</code>	Reserves a box called <code>score</code> to hold the user's number.
<code>printf("Enter score (0-100): ");</code>	Prints a prompt. No <code>\n</code> so the cursor stays on the same line, right after the colon.
<code>scanf("%d", &score);</code>	Waits for the user to type a number and press Enter. Stores it in <code>score</code> .
<code>if (score < 0 score > 100)</code>	The first guard: checks that the number is valid. <code> </code> means OR; the condition is true if <i>either</i> part is true.
<code>printf("Invalid score.\n");</code>	Only runs if the guard above is true. The <code>else if</code> chain below is skipped entirely.
<code>else if (score >= 90)</code>	Only checked if the score was valid. Is it 90 or above? If yes, run the next line. If no, move to the next <code>else if</code> .
<code>printf("Grade: A\n");</code>	Runs only for scores 90-100.
<code>else if (score >= 80)</code>	At this point we already know <code>score < 90</code> (the previous branch did not match). So this checks 80-89.
<code>else { printf("Grade: F ..."); }</code>	The final fallback. Runs if none of the conditions above were true. At this point score must be below 60.
<code>return 0;</code>	Program ends successfully.

Key point: once one `else if` condition is true, all the remaining ones are skipped. The chain exits after the first match.

Sample runs:

```
Enter score (0-100): 85    → Grade: B
Enter score (0-100): 55    → Grade: F (Fail)
Enter score (0-100): 150  → Invalid score.
```

Example 2: Day-of-week with switch

```
/* lab05_day.c */
#include <stdio.h>

int main(void) {
    int d;
    printf("Enter day number (1=Mon ... 7=Sun): ");
    scanf("%d", &d);
    switch (d) {
        case 1: printf("Monday\n"); break;
        case 2: printf("Tuesday\n"); break;
        case 3: printf("Wednesday\n"); break;
        case 4: printf("Thursday\n"); break;
        case 5: printf("Friday\n"); break;
        case 6: printf("Saturday\n"); break;
        case 7: printf("Sunday\n"); break;
        default: printf("Invalid day.\n"); break;
    }
    return 0;
}
```

Guided In-Lab Exercises

Exercise 1: Leap year checker (Period B)

A year is a leap year if it is divisible by 4, *except* years divisible by 100 are *not* leap years, *except* years divisible by 400 *are*.

Write a program that reads a year and prints “Leap year” or “Not a leap year”.

File: lab05_leap.c

Exercise 2: Menu with switch (Period B and C)

Write a program that presents a menu:

1. Circle area
2. Square area
3. Triangle area
4. Quit

Read the user’s choice (1 to 4). For choices 1 to 3, read the necessary dimensions and print the area. For 4, print “Goodbye.” Use `switch-case`.

File: lab05_menu.c

Exercise 3: Diagnostic assessment (Period B, ~10 min, ungraded)

The instructor will provide a short set of questions (5 to 8 problems) covering Weeks 1 to 5. Answer individually. Results will be used to adjust upcoming sessions and are not graded for marks.

Challenge Problem

Write a program that takes three integers and prints them in **ascending order** without using arrays or sorting functions. Use only `if-else`.

Common Pitfalls

Mistake	Effect	Fix
<code>if (x = 5)</code> instead of <code>if (x == 5)</code>	Always true! Assigns 5 to x.	Use <code>==</code> for comparison
Missing <code>break</code> in <code>switch</code>	Fall-through to next case	Add <code>break</code> after each case body
Empty <code>else</code> leaving an unhandled case	Program silently does nothing	Add <code>else</code> or <code>default</code>
Nested <code>if</code> without braces	Only the first statement is in the branch	Always use <code>{ }</code> even for one-liners

The `=` vs `==` bug is one of the most common in C. Many compilers warn about `if (x = 5)` with `-Wall`. This is why we always compile with warnings on.

Submission and Rubric

Deliverable	Filename	Points
Leap year checker	lab05_leap.c	4
Menu with switch	lab05_menu.c	6

Total: 10 points

Further Reading

- King, Ch. 5 "Selection Statements"
- K&R, Ch. 3 "Control Flow"

Lab 06: Loops I: while and do-while

Week	6
Duration	3 × 50 min (150 min)
Method	Lecture & Lab
Prerequisites	Lab 05 (conditionals, relational operators)

Why this lab matters: Imagine writing code to sum 1000 survey responses, process every line in a log file, or keep prompting a user until they enter a valid value. Writing 1000 individual statements is impractical. Loops let a small piece of code repeat as many times as needed, which is why virtually every useful program, from a data-analysis script to a web server handling requests, contains at least one loop. This lab introduces the two loop forms that check a condition before or after each step.

Time allocation

Period	Min	Activity
A (Concept)	50	5 recap · 30 while / do-while loop anatomy · 15 live demo
B (Guided practice)	50	30 guided counting/accumulation lab · 10 runtime-debugging moment · 10 debrief and pitfalls
C (Independent and wrap)	50	35 independent exercises · 10 challenge · 5 submit and Week 7 preview

Learning Outcomes

By the end of this lab, you will be able to:

1. Write `while` and `do-while` loops with correct init/condition/update structure.
 2. Use loops for counting and accumulation.
 3. Read input until a sentinel value (e.g., -1 to stop).
 4. Diagnose a misbehaving loop using `printf` -tracing.
-

Recap

In Week 5 you used `if-else` to choose between paths. Loops let you repeat a block of code: the computer can do something a thousand times as easily as once, so you just have to describe the repetition, not write it out.

Background

What Is a Loop?

In plain words: loop / iteration A loop is a way of telling the computer: “repeat this block of instructions until a certain condition is no longer true.” Each individual run of the block is called an *iteration* (one pass through the loop). Think of a song with a chorus: the chorus is the loop body, and “repeat the chorus 3 times” is the loop condition.

Loops are one of the most powerful ideas in programming. Without them, to add up 1000 numbers you would need 1000 separate lines of code. With a loop, you need 4.

while Loop Flowchart

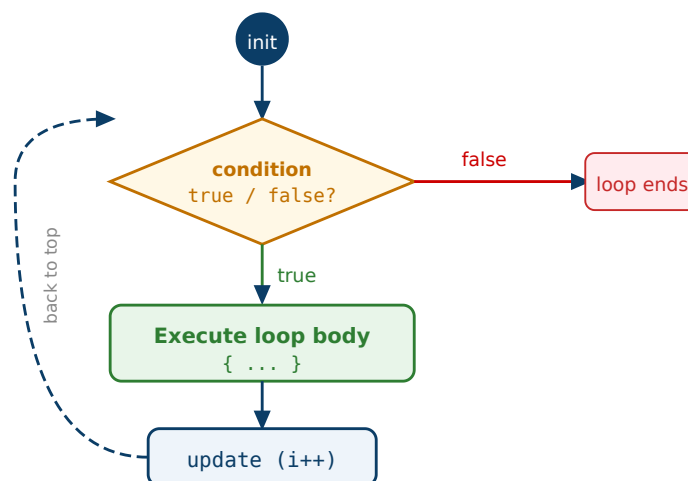


Figure 6.1. The while loop, step by step.

while Loop Syntax

```
/* init */
while (condition) {
    /* body */
    /* update */
}
```

The condition is checked **before** each iteration. If it is false from the start, the body never runs.

```
int i = 1;
while (i <= 5) {
    printf("%d\n", i);
    i++;          /* update: advance i */
}
```

do-while Loop

```
do {
    /* body */
} while (condition);  /* condition checked AFTER body */
```

The body runs **at least once**: useful for menus and input validation.

```
int choice;
do {
    printf("Enter 1-3: ");
    scanf("%d", &choice);
} while (choice < 1 || choice > 3);
printf("You chose %d\n", choice);
```

Counting and Accumulation Patterns

```
/* Sum of 1 to N */
int n, sum = 0, i = 1;
scanf("%d", &n);
while (i <= n) {
    sum += i;  /* accumulate */
    i++;
}
printf("Sum = %d\n", sum);
```

Input-Until-Sentinel

In plain words: sentinel value A *sentinel* is a special “stop” value that the user types to signal “I’m done entering data.” We use `-1` a lot because scores, counts, and prices are never negative, so `-1` cannot be a real data value. When the program sees `-1`, it knows to stop asking and start processing. Think of a sentinel like the word “STOP” at the end of an old telegram: it does not carry real information; it just means “message over.”

```
int val, total = 0, count = 0;
printf("Enter values (-1 to stop): ");
scanf("%d", &val);
while (val != -1) {
    total += val;
    count++;
    scanf("%d", &val);
}
printf("Sum=%d, Count=%d\n", total, count);
```

Under the Hood: loops as backward jumps

A `while` loop compiles to a compare and conditional jump at the top, and an unconditional **backward jump** at the bottom. The CPU jumps back and re-executes the condition check on each pass. Infinite loops happen when the backward jump always fires; you never reach the exit condition.

Worked Examples

Example 1: Number-guessing game

```
/* lab06_guess.c (outline) */
#include <stdio.h>

int main(void) {
    int secret = 42, guess;
    printf("Guess my number (1-100)!\n");
    do {
        printf("Your guess: ");
        scanf("%d", &guess);
        if (guess < secret)
            printf("Too low.\n");
        else if (guess > secret)
            printf("Too high.\n");
    } while (guess != secret);
    printf("Correct!\n");
    return 0;
}
```

Line by line

Line	What it does
<code>int secret = 42, guess;</code>	Declares two variables in one line. <code>secret</code> is initialised to 42 (the number to guess). <code>guess</code> has no value yet; <code>scanf</code> will fill it.
<code>printf("Guess my number (1-100)!\n");</code>	Prints one line of welcome text before the loop starts.
<code>do {</code>	Begins a <code>do-while</code> loop. The body runs at least once, which is exactly what we need: ask for a guess before we can check it.
<code>printf("Your guess: ");</code>	Prompts the user. No <code>\n</code> here so the cursor stays on the same line waiting for input.
<code>scanf("%d", &guess);</code>	Reads one integer from the keyboard and stores it in <code>guess</code> . The <code>&</code> gives <code>scanf</code> the address of <code>guess</code> so it can write there.
<code>if (guess < secret)</code>	Checks if the guess is too small. If yes, the next line prints "Too low." The <code>else if</code> line is skipped.
<code>else if (guess > secret)</code>	Only reaches here if the guess was NOT too small. Checks if it is too large.
<code>} while (guess != secret);</code>	The condition is checked AFTER the body. If <code>guess</code> still does not equal <code>secret</code> , the body runs again. The loop keeps going until the player finds the right number.
<code>printf("Correct!\n");</code>	Only printed once, after the loop exits, meaning <code>guess == secret</code> .

Sample run:

```
Guess my number (1-100)!
Your guess: 20
Too low.
Your guess: 60
Too high.
Your guess: 42
Correct!
```

Key insight: a `do-while` is the right choice here because we always need at least one guess before we can know whether the player has won. If you used a plain `while`, you would need to fake an initial guess to start the loop.

Example 2: Running average

```
/* lab06_avg.c (outline) */
/* Reads numbers until -1, prints count and average */
#include <stdio.h>

int main(void) {
    int val, count = 0;
    double sum = 0;
    printf("Enter integers (-1 to stop): ");
    scanf("%d", &val);
    while (val != -1) {
        sum += val;
        count++;
        scanf("%d", &val);
    }
    if (count > 0)
        printf("Average of %d values: %.2f\n", count, sum / count);
    else
        printf("No values entered.\n");
    return 0;
}
```

Guided In-Lab Exercises

Exercise 1: Sum 1 to N (Period B)

Read N from the user and print the sum $1 + 2 + \dots + N$. Also print the formula result $N \times (N+1)/2$ to verify.

File: lab06_sum.c

Exercise 2: Input validation loop (Period B)

Ask the user for a grade percentage (0 to 100). Keep asking until they enter a valid value. Then print the corresponding letter grade.

File: lab06_validate.c

Exercise 3: Guessing game (Period C)

Implement the guessing game from Example 1. After the user guesses correctly, print how many guesses it took.

File: lab06_guess.c

Runtime-Debugging Moment (Period B, ~10 min)

Scenario: A student writes the sum-to-N program but gets the wrong total.

```
int i = 0, sum = 0;
while (i <= n) {
    sum += i;
    /* forgot i++; */
}
```

This is an **infinite loop**: the program runs forever. **How to debug:** add a `printf` inside the loop:

```
printf("DEBUG: i=%d, sum=%d\n", i, sum);
```

In plain words: printf-tracing Printf-tracing means adding temporary `printf` lines inside your loop (or function) to print the values of variables on every pass. It lets you see what the program is actually doing, step by step, instead of guessing. Once the bug is fixed, delete the debug lines. This is the simplest debugging technique; you will use it every week.

Now you can see the variable values on each pass. This is your first debugging tool.

Note: this is a *runtime* bug (the code compiles fine but misbehaves). You will practice systematic debugging in Week 14.

Challenge Problem

Write a program that finds and prints all **perfect numbers** up to 1000. A perfect number equals the sum of its proper divisors (e.g., $6 = 1 + 2 + 3$).

Common Pitfalls

Mistake	Symptom	Fix
Missing update (<code>i++</code>) inside the loop	Infinite loop; program freezes	Always include the update step
Condition is false before loop starts	Loop body never executes	Check that the init value satisfies the first condition
Off-by-one: <code>while (i < n)</code> vs <code>while (i <= n)</code>	One too few or one too many iterations	Trace manually with a small value of n
Using <code>=</code> instead of <code>!=</code> in sentinel check	Loop exits immediately or never	Use <code>while (val != -1)</code>

Submission and Rubric

Deliverable	Filename	Points
Sum 1 to N	<code>lab06_sum.c</code>	3
Input validation	<code>lab06_validate.c</code>	3
Guessing game	<code>lab06_guess.c</code>	4

Total: 10 points

Further Reading

- King, Ch. 6 “Loops”
- K&R, Ch. 3 “Control Flow”
- [Debugging Tips](#)

Lab 07: Loops II: for and Nested Loops

Week	7
Duration	3 × 50 min (150 min)
Method	Lecture & Lab
Prerequisites	Lab 06 (while, do-while, accumulation)

Why this lab matters: Patterns in output, tables of data, and grid-based problems such as seating charts, pixel grids, and game boards all require a loop running inside another loop. The `for` loop is the most compact form for counted repetition and is by far the most commonly used loop in production C code. Mastering nested loops here is also direct preparation for two-dimensional arrays, image processing, and matrix arithmetic in later labs.

Time allocation

Period	Min	Activity
A (Concept)	50	5 recap · 30 for loop, nested loops, break/continue · 15 live demo
B (Guided practice)	50	40 guided pattern lab · 10 debrief and pitfalls
C (Independent and wrap)	50	20 independent patterns · 15 pre-midterm mixed review · 5 submit and Week 8 preview

Learning Outcomes

By the end of this lab, you will be able to:

1. Write `for` loops and explain why they are preferred when the iteration count is known.
2. Write nested loops to generate 2-D output (multiplication tables, patterns).
3. Use `break` to exit a loop early and `continue` to skip to the next iteration.
4. Solve problems using a combination of loops and conditionals (pre-midterm review).

Recap

In Week 6 you used `while` and `do-while` for repetition. The `for` loop covers the same ground but bundles init, condition, and update into one line, making it cleaner when you know how many times you want to loop.

Background

for Loop Anatomy

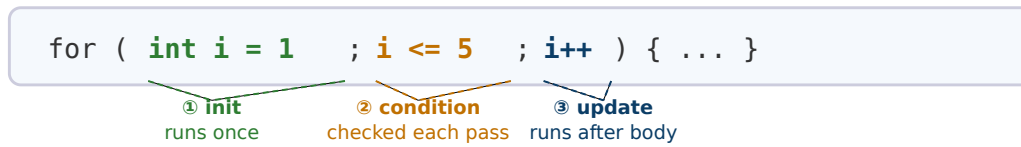


Figure 7.1. Anatomy of a for loop.

```
for (int i = 1; i <= 5; i++) {  
    printf("%d\n", i);  
}
```

When to choose for : use it when the number of iterations is known before the loop starts.

When to choose while : use it when you are waiting for an event (e.g., sentinel input).

Nested Loops: a Grid of Iterations

	col 1	col 2	col 3	col 4
row 1	1,1	1,2	1,3	1,4
row 2	2,1	2,2	2,3	2,4
row 3	3,1	3,2	3,3	3,4

inner loop (col) runs 4 times per row → 3 x 4 = 12 total

Figure 7.2. Nested loops as a grid of iterations.

```
for (int row = 1; row <= 3; row++) {  
    for (int col = 1; col <= 4; col++) {  
        printf("%4d", row * col);  
    }  
    printf("\n");  
}
```

The inner loop runs fully for each step of the outer loop. A 3x4 nested loop does 3 x 4 = 12 iterations total.

Analogy: think of a washing machine's program selector (outer loop) combined with the drum spin cycles (inner loop). For each wash program (hot, warm, cold), the drum spins a fixed number of times. You do not choose the next wash program until all spins for the current one are done. The inner loop always finishes completely before the outer loop advances.

break and continue

In plain words: **break** `break` immediately stops the loop and jumps to the first line *after* the closing `}`. Use it when you have found what you were looking for and there is no reason to keep iterating. For example, once you find a match in a list, you **break** out of the search loop. Important: **break** only exits the *innermost* loop. If you are inside a nested loop, the outer loop keeps running.

In plain words: **continue** `continue` skips the rest of the current loop body and goes straight to the update step (`i++`), then re-checks the condition. Use it to skip an individual iteration without exiting the loop entirely. For example, to skip even numbers: if `i` is even, **continue**; otherwise process it.

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) break;           /* exit loop when i reaches 5 */  
    if (i % 2 == 0) continue;    /* skip even numbers */  
    printf("%d\n", i);          /* prints 1, 3 */  
}
```

Under the Hood: nested loops and work

If the outer loop runs N times and the inner loop runs M times, the body executes $N \times M$ times. Doubling N doubles the total work. This is why nested loops can be slow for large inputs, a topic you will study in depth in **Algorithms and Data Structures**.

Worked Examples

Example 1: Multiplication table (n x n)

```
/* lab07_table.c */
#include <stdio.h>

int main(void) {
    int n;
    printf("Print n x n table. n = ");
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            printf("%4d", i * j);
        }
        printf("\n");
    }
    return 0;
}
```

Line by line

Line	What it does
<code>int n;</code>	Declares a variable to hold the table size the user will type.
<code>printf("Print n x n table. n = ");</code>	Prompts the user. No <code>\n</code> so the cursor waits on the same line.
<code>scanf("%d", &n);</code>	Reads the user's number into <code>n</code> .
<code>for (int i = 1; i <= n; i++)</code>	Outer loop. <code>i</code> represents the current row . Starts at 1, goes up to and including <code>n</code> , increments by 1 after each row.
<code>for (int j = 1; j <= n; j++)</code>	Inner loop. <code>j</code> represents the current column . For <i>each value of i</i> , this loop runs all the way from 1 to <code>n</code> before <code>i</code> advances.
<code>printf("%4d", i * j);</code>	Prints the product <code>i * j</code> in a 4-character-wide field. <code>%4d</code> aligns numbers in columns so they line up neatly. No <code>\n</code> here: we stay on the same row until the inner loop is done.
<code>printf("\n");</code>	After the inner loop finishes one complete row of numbers, move to the next line. This <code>printf</code> belongs to the <i>outer</i> loop.

Sample output (n = 4):

1	2	3	4
2	4	6	8
3	6	9	12
4	8	12	16

Notice how `i = 2` (row 2) gives `2*1=2`, `2*2=4`, `2*3=6`, `2*4=8`. Each row is just the multiplication table for that row number.

Example 2: Triangle pattern

```
/* lab07_triangle.c */
#include <stdio.h>

int main(void) {
    int rows;
    printf("Number of rows: ");
    scanf("%d", &rows);
    for (int i = 1; i <= rows; i++) {
        for (int j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }
    return 0;
}
```

Sample output (rows = 4):

```
*
* *
* * *
* * * *
```

Guided In-Lab Exercises

Exercise 1: Right-aligned triangle (Period B)

Print a triangle that is **right-aligned** (spaces on the left):

```
   *
  * *
 * * *
* * * *
```

Read the number of rows from the user.

File: `lab07_rtriangle.c`

Exercise 2: Hollow rectangle (Period B)

Print a hollow rectangle of `*` characters given width and height:

```
* * * * *
*       *
*       *
* * * * *
```

File: `lab07_hollow.c`

Exercise 3: Pre-midterm review problems (Period C)

Solve these mixed problems (one file per task):

- **Review A:** Read N integers and print the largest and smallest.
- **Review B:** Print all integers from 1 to 100 that are divisible by 3 or 5 (but not both).
- **Review C:** Read a positive integer and print its digits in reverse order.

Files: `lab07_rev_a.c`, `lab07_rev_b.c`, `lab07_rev_c.c`

Challenge Problem

Print a diamond pattern:

```
  *
 ***
*****
*****
 *****
  ***
   *
```

The number of rows in the top half is read from the user.

Common Pitfalls

Mistake	Symptom	Fix
Modifying loop variable inside the body	Loop runs wrong number of times	Keep the update in the <code>for</code> header
Off-by-one in nested loop bounds	One extra or missing row/column	Trace with small n (e.g., 3) on paper first
<code>break</code> only exits the innermost loop	Outer loop continues	Use a flag variable if you need to exit multiple levels
Infinite <code>for</code> loop with empty condition <code>for(;;)</code>	Never terminates without <code>break</code>	Ensure a condition or use <code>break</code>

Submission and Rubric

Deliverable	Filename	Points
Right-aligned triangle	<code>lab07_rtriangle.c</code>	3
Hollow rectangle	<code>lab07_hollow.c</code>	3
Review problems (3 total)	<code>lab07_rev_a.c</code> , <code>_b.c</code> , <code>_c.c</code>	4

Total: 10 points

Further Reading

- King, Ch. 6 “Loops”
- K&R, Ch. 3 “Control Flow”
- Practice the review problems. **Midterm is next week** (Week 8, covers Weeks 1 to 7).

Lab 08: Midterm: Live In-Lab Coding

Week	8
Duration	150 min (single continuous block)
Method	Examination (live coding)
Scope	Weeks 1 to 7 (variables, I/O, operators, conditionals, loops)
Weight	20% of final grade

Time allocation

Phase	Min	Activity
Setup	10	Rules, seat assignment, confirm environment works
Live coding	120	Several independent tasks (see format below)
Submission	20	Upload/submit files; buffer for technical issues

Exam Format

- **Open-book:** you may use your compiler, reference sheets, and the course lab pages.
- **Restricted network:** no internet browsing, no messaging, no AI tools.
- **Invigilated:** work independently; talking to classmates is not permitted.
- **Partial credit per task:** each task is graded independently. Completing 3 of 4 tasks correctly earns 75% of the exam mark. A partially correct solution earns more than a blank answer.

Typical Task Structure

The exam contains **3 to 5 short tasks**, each solvable in 20 to 40 minutes. Each task:

- Has a clear input specification and expected output.
- Is self-contained (you do not need the output of task 1 for task 2).
- Is graded on: Compiles, Correct output, Code style, Approach.

Submission

- Submit each task as a separate `.c` file: `m1.c`, `m2.c`, etc.
- Files are submitted through the LMS or a designated folder before time is up.
- Do not submit files that fail to compile; fix the error first even if the output is wrong.

Learning Outcomes Being Assessed

This exam assesses all outcomes from Weeks 1 to 7:

1. Write a syntactically correct C program from scratch.
 2. Use `int`, `double`, `char` variables and `printf` / `scanf` for I/O.
 3. Apply arithmetic, relational, and logical operators correctly.
 4. Use `if-else` and `switch-case` to branch based on conditions.
 5. Write `while`, `do-while`, and `for` loops for counting and accumulation.
 6. Read error messages and fix compilation errors.
-

What to Bring

- Your lab machine (or a laptop with gcc installed).
 - A printed or offline copy of the reference sheet (provided by instructor).
 - No notes on electronic devices other than your own source files from labs.
-

Sample Practice Problems

These are **not** the actual exam questions, but they are representative in scope and style.

Practice 1: Read N and print the sum of odd numbers from 1 to N.

Practice 2: Read a temperature in Celsius and classify it:

- Below 0: "Freezing"
- 0 to 15: "Cold"
- 16 to 30: "Comfortable"
- Above 30: "Hot"

Practice 3: Read numbers until the user enters 0. Print the count of positive numbers, the count of negative numbers, and the average of all entered values.

Practice 4: Print the numbers 1 to 50, but for multiples of 3 print "Fizz", for multiples of 5 print "Buzz", and for multiples of both print "FizzBuzz".

Exam Rubric (per task)

Criterion	Points
Program compiles without errors	2
Correct output for sample test case	4
Correct output for hidden test case	2
Code style (names, indentation, comments)	1
Appropriate algorithm / no unnecessary code	1
Task total	10

If the program does not compile, style and algorithm points are not awarded. Fix compilation errors before moving on.

After the Exam

Results and feedback will be returned within one week. The **project** (30%) will be assigned around Weeks 11 to 12; the **final exam** (30%) is your Week-15 project demo plus individual oral defense.

Further Reading

- Review [Lab 01](#) through [Lab 07](#).
- [Debugging Tips](#)

Lab 09: Functions I: Basics

Week	9
Duration	3 × 50 min (150 min)
Method	Lecture & Lab
Prerequisites	Lab 07 (all control flow), midterm experience

Why this lab matters: Professional programs are never written as one long block of code. They are organized into named, reusable functions, in the same way a recipe is broken into steps that can be reused across many dishes. Functions let you write code once, test it in isolation, and call it from anywhere in your program. Every library you will ever use, from the C standard library to a graphics toolkit to a network API, is a collection of functions. This lab teaches you to write and use your own.

Time allocation

Period	Min	Activity
A (Concept)	50	5 recap midterm · 30 functions: definition, call, prototype, return · 15 live demo
B (Guided practice)	50	40 guided mini-library lab · 10 debrief and pitfalls
C (Independent and wrap)	50	35 independent exercises · 10 challenge · 5 submit and Week 10 preview

Learning Outcomes

By the end of this lab, you will be able to:

1. Define a function with a return type, parameter list, and body.
2. Declare a function prototype before `main` and define it after.
3. Call a function and use its return value.
4. Explain why breaking code into functions makes programs easier to write, test, and read.

Recap

Until now, all your code has been inside `main`. As programs grow, a single large `main` becomes hard to read and debug. Functions let you **name a piece of logic**, call it anywhere,

and test it independently.

Background

What Is a Function?

In plain words: function A function is a named block of code that performs one specific task. You write it once and can call it (run it) as many times as you like, from anywhere in your program. Think of a function like a **recipe**: once you have written the recipe for “make toast,” you can follow it every morning without re-writing it. The recipe takes inputs (bread, butter) and produces an output (toast). In C, inputs are called *parameters* and the output is called the *return value*.

Using functions has three big benefits:

1. **No repetition.** Write once, call many times.
2. **Easier to test.** You can test each recipe separately.
3. **Easier to read.** `max(a, b)` is far clearer than an in-line if-else every time.

In plain words: parameter vs argument A *parameter* is the variable name in the function definition: `int max(int a, int b)` has parameters `a` and `b`. An *argument* is the actual value you pass when you call it: `max(10, 25)` passes arguments `10` and `25`. C copies each argument into its corresponding parameter. Changing the parameter inside the function does NOT change the original variable in the caller.

In plain words: return value The *return value* is what the function sends back to the caller. `return 25;` hands the number 25 back so the caller can use it. A function can only return one value. If you need to change multiple variables, use pointers (Week 13) or restructure your design.

Function Call and Return: the Jump

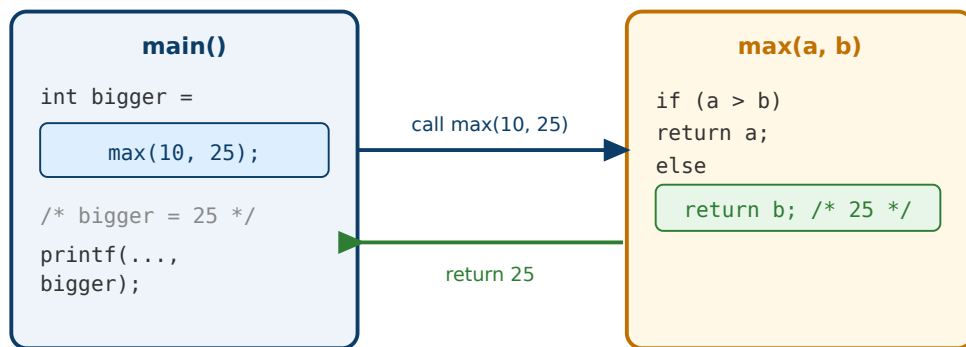


Figure 9.1. A function call and its return.

Function Anatomy

```
/* PROTOTYPE (declaration): goes before main */
int max(int a, int b);

int main(void) {
    int bigger = max(10, 25);    /* CALL */
    printf("Max = %d\n", bigger);
    return 0;
}

/* DEFINITION: goes after main */
int max(int a, int b) {
    if (a > b) return a;
    else      return b;
}
```

Part	Purpose
Return type (<code>int</code>)	The type of value the function sends back
Name (<code>max</code>)	How you call it
Parameters (<code>int a, int b</code>)	Input values the caller provides
Body { ... }	What the function does
<code>return</code>	Sends a value back to the caller

void Functions

```
void print_line(int n) { /* no return value */  
    for (int i = 0; i < n; i++) printf("-");  
    printf("\n");  
}
```

Call with: `print_line(30);` No `return` needed (or write `return;` with no value).

Under the Hood: function calls

When you call a function, the CPU:

1. Pushes the arguments and return address onto the **call stack** (a region of RAM).
2. Jumps to the function's first instruction.
3. When `return` is reached, pops the stack and jumps back.

Each function invocation gets its own **stack frame**, its own copy of local variables. The call stack is finite; calling functions recursively many thousands of times can overflow it. You will study the stack in depth in **System Programming**.

Worked Examples

Example 1: Mini math library

```
/* lab09_mathlib.c */
#include <stdio.h>

int max(int a, int b);
int min(int a, int b);
int is_prime(int n);

int main(void) {
    printf("max(7,12) = %d\n", max(7, 12));
    printf("min(7,12) = %d\n", min(7, 12));
    printf("is_prime(17) = %d\n", is_prime(17));
    printf("is_prime(15) = %d\n", is_prime(15));
    return 0;
}

int max(int a, int b) { return (a > b) ? a : b; }
int min(int a, int b) { return (a < b) ? a : b; }

int is_prime(int n) {
    if (n < 2) return 0;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return 0;
    }
    return 1;
}
```

Line by line

The three prototypes at the top tell the compiler: “these functions exist; trust me.” The definitions at the bottom provide the actual bodies.

Line	What it does
<code>int max(int a, int b);</code>	Prototype for <code>max</code> . Declares that <code>max</code> takes two <code>int</code> parameters and returns one <code>int</code> . No body here; the semicolon ends the declaration.
<code>int min(int a, int b);</code>	Prototype for <code>min</code> . Same shape.
<code>int is_prime(int n);</code>	Prototype for <code>is_prime</code> . Takes one <code>int</code> , returns one <code>int</code> (1 = yes, 0 = no).
<code>printf("max(7,12) = %d\n", max(7, 12));</code>	Calls <code>max(7, 12)</code> . The call happens inside <code>printf</code> 's argument list. The return value (12) is plugged into <code>%d</code> and printed. <code>main</code> never sees a temporary variable; the value goes straight to <code>printf</code> .
<code>int max(int a, int b) { return (a > b) ? a : b; }</code>	Definition of <code>max</code> . <code>(a > b) ? a : b</code> is the ternary operator: "if <code>a > b</code> , give <code>a</code> ; otherwise give <code>b</code> ." This is equivalent to the if-else version but shorter.
<code>if (n < 2) return 0;</code>	Inside <code>is_prime</code> : any number below 2 is not prime. Return <code>0</code> (false) immediately.
<code>for (int i = 2; i * i <= n; i++)</code>	Only check divisors up to the square root of <code>n</code> . If <code>n</code> has no divisors up to <code>sqrt(n)</code> , it is prime. This is a classic optimization.
<code>if (n % i == 0) return 0;</code>	If any divisor divides <code>n</code> evenly (remainder = 0), <code>n</code> is not prime. Return <code>0</code> and exit.
<code>return 1;</code>	If the loop finished without finding any divisor, <code>n</code> is prime. Return <code>1</code> (true).

Expected output:

```
max(7,12) = 12
min(7,12) = 7
is_prime(17) = 1
is_prime(15) = 0
```

Example 2: factorial function

```
int factorial(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) result *= i;
    return result;
}
```

Guided In-Lab Exercises

Exercise 1: Extend the math library (Period B)

Add these functions to the example above:

- `int gcd(int a, int b)` (greatest common divisor using Euclid's algorithm).
- `double power(double base, int exp)` (base raised to the integer exponent, without `<math.h>`).

Test each function from `main` with at least two inputs.

File: `lab09_mathLib.c`

Exercise 2: Input validator as a function (Period B and C)

Write a function `int read_positive(void)` that:

1. Prompts "Enter a positive integer: "
2. Reads an integer
3. If it is 0 or negative, prints "Invalid, try again." and loops
4. Returns the valid positive integer

Use this function from `main` to read two values, then print their sum.

File: `lab09_validator.c`

Exercise 3: Modular grade printer (Period C)

Write three functions:

- `double read_score(const char* label)` prints `label` , reads a double 0 to 100.
- `char grade_letter(double score)` returns the letter grade ('A' to 'F').
- `void print_report(double s1, double s2, double s3)` prints each score with its grade and the average.

File: `lab09_grades.c`

Challenge Problem

Write a function `int count_digits(int n)` that counts how many digits the integer `n` has. Write a function `int reverse_num(int n)` that reverses the digits of `n`. Test both with several inputs including 0 and negative numbers.

Common Pitfalls

Mistake	Symptom	Fix
Calling a function before its prototype	Warning: implicit declaration	Write the prototype above <code>main</code>
Wrong return type	Compiler warning; truncated value	Match the return type to what you <code>return</code>
Expecting a function to modify a caller variable	Variable unchanged in <code>main</code>	C passes by value. Return the new value or use a pointer (Week 13).
Missing <code>return</code> in non-void function	Undefined behaviour	All paths must reach a <code>return</code>

Submission and Rubric

Deliverable	Filename	Points
Extended math library	<code>lab09_mathlib.c</code>	4
Input validator	<code>lab09_validator.c</code>	3
Modular grade printer	<code>lab09_grades.c</code>	3

Total: 10 points

Further Reading

- King, Ch. 9 "Functions"
- K&R, Ch. 4 "Functions and Program Structure"

Lab 10: Functions II: Scope & Recursion

Week	10
Duration	3 × 50 min (150 min)
Method	Lecture & Lab
Prerequisites	Lab 09 (function definition, call, prototype)

Why this lab matters: Understanding variable scope prevents a whole category of bugs that beginners encounter when they expect a change inside a function to affect a variable outside it. Recursion, where a function calls itself, is the natural solution to a set of real problems: traversing a file-system tree, parsing nested data, and implementing algorithms like merge sort. Tracing recursion on paper also builds your mental model of the call stack, which is essential background for debugging and for system-level programming.

Time allocation

Period	Min	Activity
A (Concept)	50	5 recap · 30 scope, pass-by-value, recursion basics · 15 live demo
B (Guided practice)	50	40 guided recursive lab · 10 debrief and pitfalls
C (Independent and wrap)	50	35 independent exercises · 10 challenge · 5 submit and Week 11 preview

Learning Outcomes

By the end of this lab, you will be able to:

1. Distinguish between local and global scope and explain why global variables are avoided.
2. Explain pass-by-value: why changing a parameter inside a function does not change the caller's variable.
3. Write a simple recursive function with a clear base case and recursive case.
4. Trace a recursive call on paper using a call-stack diagram.

Recap

In Week 9 you learned to define, prototype, and call functions. This week you go deeper: how does scope work, why can't you change a caller's variable from inside a function, and what is recursion, a function that calls itself?

Background

Key Terms

In plain words: scope The *scope* of a variable is the region of the program where that variable exists and can be used. A variable declared inside a function (between `{` and `}`) is *local* to that function. It is created when the function starts and destroyed when the function returns. A variable declared outside all functions is *global*: every function can see it. Prefer local variables; global variables make programs hard to reason about because any function can change them at any time.

In plain words: pass-by-value When you call a function and pass a variable, C makes a **copy** of the value and puts it in the parameter. The function works on the copy. If the function changes the copy, the original variable in the caller is NOT affected. This is called pass-by-value. Think of faxing a document: the recipient gets a copy; writing on it does not change your original. To let a function change a caller's variable, you pass a pointer (Week 13).

In plain words: recursion A function is *recursive* if it calls itself. Each call works on a smaller version of the problem. You must define:

1. A **base case**: a simple situation where the function returns without calling itself.
2. A **recursive case**: a call that moves one step closer to the base case.

Think of Russian nesting dolls: opening a doll reveals a smaller doll inside. You keep opening until you reach the smallest doll that cannot open further (base case). Then you close them back up one at a time, going back to the original.

In plain words: stack frame Every function call gets its own private workspace in a region of memory called the *call stack*. This workspace is called a *stack frame*. It holds the function's local variables and the address to return to when done. When a recursive function calls itself, a new frame is pushed on top. Frames are popped off as each call returns. Calling too deeply fills the stack and crashes the program (stack overflow).

Think of a stack of plates. Each function call adds one plate to the top. Each return removes the top plate. You can only ever work on the top plate.

Local Scope and Pass-by-Value

Every variable declared inside a function is **local** to that function. It exists only while the function is running; calling the function again gives a fresh copy.

```
void increment(int x) {
    x = x + 1;          /* changes local x, NOT the caller's copy */
    printf("%d\n", x); /* prints caller's value + 1 */
}

int main(void) {
    int a = 5;
    increment(a);       /* a is still 5 here */
    printf("%d\n", a); /* prints 5 */
    return 0;
}
```

This is **pass-by-value**: C copies the value into the parameter. To modify the caller's variable, you need pointers (introduced in Week 13).

Recursion and the Call Stack

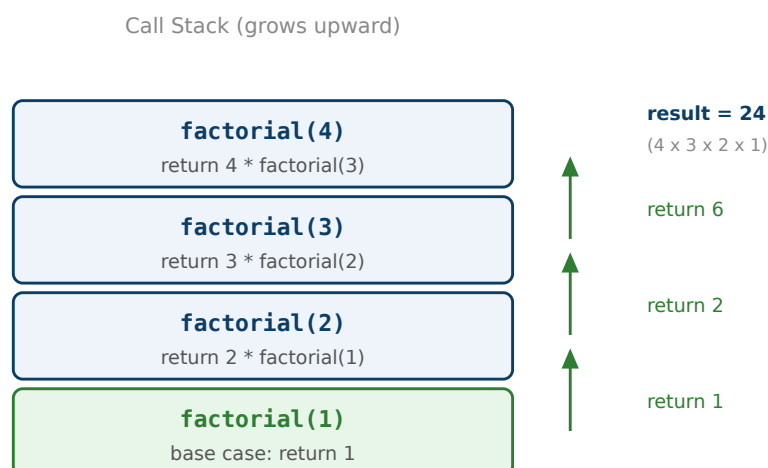


Figure 10.1. The call stack for *factorial(4)*.

A function is **recursive** if it calls itself. Every recursive function needs:

1. A **base case**: a condition where it returns without calling itself.
2. A **recursive case**: a call that moves toward the base case.

```
int factorial(int n) {  
    if (n <= 1) return 1;          /* base case */  
    return n * factorial(n - 1);  /* recursive case */  
}
```

Under the Hood: the call stack and recursion

Each recursive call pushes a new **stack frame** onto the call stack. `factorial(4)` pushes 4 frames before any return. Deep recursion (thousands of levels) can **overflow the stack**, crashing the program. In **System Programming** you will learn the exact size of the call stack and techniques to work around its limits. For this course, keep recursion depth reasonable (less than a few thousand).

Worked Examples

Example 1: Recursive countdown

```
/* lab10_countdown.c */  
#include <stdio.h>  
  
void countdown(int n) {  
    if (n <= 0) {          /* base case */  
        printf("Go!\n");  
        return;  
    }  
    printf("%d...\n", n);  
    countdown(n - 1);      /* recursive case */  
}  
  
int main(void) {  
    countdown(5);  
    return 0;  
}
```

Line by line

Line	What it does
<code>void countdown(int n)</code>	Function header. Return type <code>void</code> (nothing returned). Parameter <code>n</code> is a local copy of whatever the caller passes.
<code>if (n <= 0)</code>	Base case check. If <code>n</code> has reached zero or below, stop recurring.
<code>printf("Go!\n");</code> <code>return;</code>	The base case action. Prints "Go!" then returns to whoever called this invocation. <code>return;</code> with no value is valid in a <code>void</code> function.
<code>printf("%d...\n", n);</code>	Print the current value of <code>n</code> . This happens BEFORE the recursive call, so numbers count down: 5, 4, 3...
<code>countdown(n - 1);</code>	Recursive case. Call <code>countdown</code> again with <code>n</code> minus 1. This pushes a new frame. The current frame waits here until the recursive call returns.

Trace for `countdown(3)` :

```
countdown(3): print "3...", then call countdown(2)
  countdown(2): print "2...", then call countdown(1)
    countdown(1): print "1...", then call countdown(0)
      countdown(0): print "Go!", return
    (countdown(1) returns)
  (countdown(2) returns)
(countdown(3) returns)
```

Expected output (`countdown(5)`):

```
5...
4...
3...
2...
1...
Go!
```

Example 2: Recursive sum of digits

```
int digit_sum(int n) {
    if (n < 10) return n;           /* single digit: base case */
    return n % 10 + digit_sum(n / 10); /* peel last digit */
}
```

Guided In-Lab Exercises

Exercise 1: Recursive power (Period B)

Write `double power(double base, int exp)` recursively:

- Base case: `exp == 0` gives `1.0`
- Recursive case: `base * power(base, exp - 1)`

Test it and compare with your iterative version from Week 9.

File: `lab10_power.c`

Exercise 2: Refactor a Week-7 program (Period B and C)

Take your `lab07_rtriangle.c` (right-aligned triangle) and rewrite it using two functions:

- `void print_spaces(int n)` prints `n` spaces
- `void print_stars(int n)` prints `n` star characters and a newline
- `void print_triangle(int rows)` calls the above

File: `lab10_triangle.c`

Exercise 3: Fibonacci (Period C)

Write `int fib(int n)` that returns the `n`th Fibonacci number recursively:

- `fib(0) = 0, fib(1) = 1, fib(n) = fib(n-1) + fib(n-2)`

Print `fib(0)` through `fib(10)` and note how slow it becomes for larger `n`. (Why? Each call spawns two more calls: exponential growth.)

File: `lab10_fib.c`

Challenge Problem

Write a recursive function `int gcd(int a, int b)` using the Euclidean algorithm:

- Base case: `b == 0` gives `a`
- Recursive case: `gcd(b, a % b)`

Compare with an iterative version and verify they give the same results.

Common Pitfalls

Mistake	Symptom	Fix
Missing base case	Stack overflow crash	Every recursive function must have a base case that terminates
Base case is never reached	Same as above	Ensure the recursive call moves toward the base case
Expecting pass-by-value to modify caller	Variable unchanged	Return the modified value, or use pointers (Week 13)
Global variable to share state	Works but fragile	Pass data via parameters and return values

Submission and Rubric

Deliverable	Filename	Points
Recursive power	lab10_power.c	3
Refactored triangle	lab10_triangle.c	3
Fibonacci	lab10_fib.c	4

Total: 10 points

Further Reading

- King, Ch. 9 “Functions”
- K&R, Ch. 4 “Functions and Program Structure”

Lab 11: Arrays I: One-Dimensional Arrays

Week	11
Duration	3 × 50 min (150 min)
Method	Lecture & Lab
Prerequisites	Lab 10 (functions, loops)

Why this lab matters: Almost every meaningful program handles a collection of values: a list of exam scores, a series of temperature readings, a set of product prices. Without arrays, storing ten scores would require ten separate variable names; with an array, `int scores[10]` handles all ten at once and a loop processes them in two lines. Arrays are also the gateway to every data structure, such as strings, tables, and images, that you will meet in this and future courses.

Time allocation

Period	Min	Activity
A (Concept)	50	5 recap · 30 array declaration, indexing, traversal · 15 live demo
B (Guided practice)	50	40 guided stats lab · 10 debrief and pitfalls
C (Independent and wrap)	50	35 independent exercises · 10 challenge · 5 submit and Week 12 preview and project announcement

Learning Outcomes

By the end of this lab, you will be able to:

1. Declare and initialize one-dimensional arrays.
 2. Read and write array elements using index-based loops.
 3. Implement common array operations: sum, average, minimum, maximum, linear search.
 4. Explain why array indexing starts at 0 and what out-of-bounds access causes.
 5. Recognize that a C string is a one-dimensional array of `char` ending in `'\0'`.
 6. Put a one-dimensional array in order using bubble sort.
-

Recap

Until now, your programs handled a fixed number of variables (`a` , `b` , `c`). An array lets you store a **collection** of values of the same type under one name and access them by position.

Background

What Is an Array?

In plain words: array An array is a row of boxes in memory, all holding the same type, all sharing one name. You access each box by its position number, called the *index*. Think of a row of numbered lockers in a school corridor: all lockers have the same size (type), they are all in a row (contiguous memory), and you find any locker instantly by its number. Locker #0 is the first one, not locker #1.

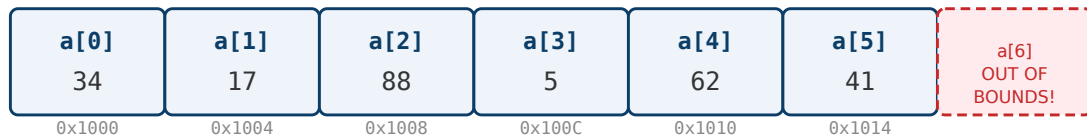
Without arrays, storing 100 exam scores would require 100 separate variable names. With an array: `int scores[100];` creates all 100 boxes at once.

In plain words: index The *index* is the number you put inside the square brackets `[]` to pick one element. In C, arrays always start at index **0**, not 1. An array declared `int a[5]` has elements `a[0]` , `a[1]` , `a[2]` , `a[3]` , `a[4]` . The last valid index is always `size - 1` , which here is 4. Forgetting this and using `a[5]` is a very common bug.

In plain words: out-of-bounds access Reading or writing `a[5]` when the array only has indices 0 to 4 is called an *out-of-bounds access*. The C compiler does NOT check for this automatically. The program reads (or writes) whatever bytes happen to be at that memory address, which could be another variable, a return address, or anything else. The result is unpredictable: the program might crash, produce garbage output, or appear to work fine until something worse happens. Always make sure your loop stops at `i < n` .

Arrays as Contiguous Memory

```
int a[6];
```



Each element is 4 bytes (sizeof int). $a[i]$ lives at address: $\text{start} + i \times 4$.

$a[6]$ reads random memory: the CPU does not check bounds for you!

Figure 11.1. A one-dimensional array in memory.

Declaring and Initializing Arrays

```
int scores[5]; /* uninitialized: values are garbage */
int primes[5] = {2, 3, 5, 7, 11}; /* initialized */
int zeros[10] = {0}; /* all elements set to 0 */
double data[] = {1.5, 2.5, 3.5}; /* compiler counts: size = 3 */
```

Traversal with a Loop

```
int a[5] = {10, 20, 30, 40, 50};
for (int i = 0; i < 5; i++) {
    printf("a[%d] = %d\n", i, a[i]);
}
```

Common Operations

```
/* Sum and average */
int sum = 0;
for (int i = 0; i < n; i++) sum += a[i];
double avg = (double)sum / n;

/* Minimum */
int min = a[0];
for (int i = 1; i < n; i++)
    if (a[i] < min) min = a[i];

/* Linear search: returns index or -1 */
int search(int a[], int n, int target) {
    for (int i = 0; i < n; i++)
        if (a[i] == target) return i;
    return -1;
}
```

Arrays as Function Parameters

```
void print_array(int a[], int n) { /* array size is lost: pass n separately */
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}
```

Always pass the **size** `n` as a separate parameter.

Under the Hood: contiguous memory

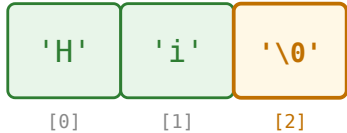
An array is a contiguous block of memory. `a[i]` is equivalent to `*(a + i)`: start of the array plus `i` times the element size. This is why `a[5]` in a 5-element array reads random memory: the CPU computes `start + 5 × 4 bytes` and fetches whatever bytes happen to be there. Out-of-bounds access is a very common source of crashes and security vulnerabilities. The computer does not check; you must.

Strings: A First Look

Until now the arrays in your programs have held `int` or `double` values. The same idea works with `char`: a 1-D array of characters. C adds exactly one rule: the last real character must be followed by `'\0'` (the null terminator, ASCII code 0), which marks where the text ends.

In plain words: string (first look) A *string* in C is not a special type. It is a one-dimensional array of `char` with one rule: the last real character is followed by `'\0'` (the null terminator, ASCII 0), which marks “the text ends here.” Because it is just an array, you index it like any other: `word[0]` is the first character. The full toolkit for strings (reading them, `strlen`, `strcpy`, `strcmp`, and arrays of strings) comes next week in Lab 12.

```
char word[] = "Hi";
```



3 bytes: 2 letters + the null terminator '\0'

The diagram shows three adjacent boxes representing memory cells. The first box is green and contains the character 'H' with the index [0] below it. The second box is green and contains the character 'i' with the index [1] below it. The third box is yellow and contains the null terminator '\0' with the index [2] below it.

Figure 11.2. A string is a one-dimensional char array ending in `'\0'`.

Because a string is just a 1-D array, you can use it exactly like one:

```

char word[] = "Hi";          /* a 1-D char array: 'H', 'i', '\0' */
printf("%s\n", word);        /* print the whole string: Hi      */
printf("%c\n", word[0]);     /* print one element:   H        */

for (int i = 0; word[i] != '\0'; i++) /* walk it like any array */
    printf("%c ", word[i]);          /* H i                      */
printf("\n");

```

Two rules to remember now:

1. Index a string like any array: `word[0]` is the first character.
2. Always leave room for `'\0'`: a 5-letter word needs at least 6 bytes.

Full string coverage (reading strings, `<string.h>`, arrays of strings) is in [Lab 12: Arrays II and Strings](#).

Sorting: A First Look

Beyond *finding* a value (linear search above), another very common array task is putting the values *in order*. The simplest method to learn first is bubble sort.

In plain words: sorting (bubble sort) To *sort* an array is to rearrange its elements into order (smallest to largest here). The simplest method is *bubble sort*: walk left to right comparing each pair of neighbours, and swap them whenever they are out of order. After one full pass the largest value has “bubbled” to the end. Repeat over the shrinking front of the array until a pass makes no swaps. Bubble sort is slow on large arrays but easy to understand, which is why it is the first sort most people learn. Faster methods, and how to measure their speed (Big-O), come in a later Data Structures course.

Compare neighbours; swap if out of order.

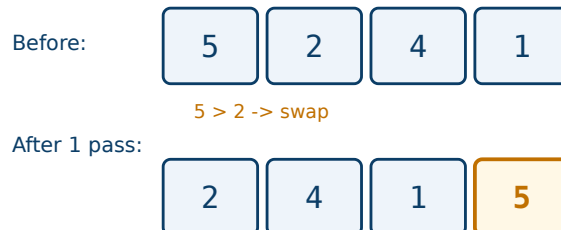


Figure 11.3. One pass of bubble sort moves the largest remaining value to the end.

```

/* Bubble sort: put a[0..n-1] in ascending order */
void bubble_sort(int a[], int n) {
    for (int pass = 0; pass < n - 1; pass++) {
        for (int i = 0; i < n - 1 - pass; i++) {
            if (a[i] > a[i + 1]) { /* out of order? swap the pair */
                int tmp = a[i];
                a[i] = a[i + 1];
                a[i + 1] = tmp;
            }
        }
    }
}

```

Line	What it does
<pre> for (int pass = 0; pass < n - 1; pass++) </pre>	Outer loop: one pass per iteration. After each pass the next-largest value has settled at the end, so at most $n-1$ passes are needed for n elements.
<pre> for (int i = 0; i < n - 1 - pass; i++) </pre>	Inner loop: compare each neighbouring pair. The $- \text{pass}$ shrinks the range each time because the last pass elements are already in their final place.
<pre> if (a[i] > a[i + 1]) </pre>	If the left neighbour is bigger than the right one, they are out of order: swap them.
<pre> int tmp = a[i]; a[i] = a[i+1]; a[i+1] = tmp; </pre>	Classic three-step swap using a temporary variable. Writing <code>a[i] = a[i+1]; a[i+1] = a[i];</code> would overwrite <code>a[i]</code> before you copy it, so <code>tmp</code> is essential.

Trace of `bubble_sort` on `{5, 2, 4, 1}`:

```

start:  5 2 4 1
pass 1: 2 4 1 5      (largest, 5, bubbles to the end)
pass 2: 2 1 4 5
pass 3: 1 2 4 5      (sorted)

```

You will use bubble sort to alphabetize names in [Lab 12](#) and to rank student records in [Lab 13](#).

Worked Examples

Example 1: Statistics on an array

```
/* lab11_stats.c (outline) */
#include <stdio.h>
#define N 8

int main(void) {
    int a[N] = {34, 17, 88, 5, 62, 41, 99, 23};
    int sum = 0, min = a[0], max = a[0];

    for (int i = 0; i < N; i++) {
        sum += a[i];
        if (a[i] < min) min = a[i];
        if (a[i] > max) max = a[i];
    }

    printf("Sum: %d\n", sum);
    printf("Avg: %.2f\n", (double)sum / N);
    printf("Min: %d\n", min);
    printf("Max: %d\n", max);
    return 0;
}
```

Line by line

Line	What it does
<pre>#define N 8</pre>	A named constant. Every occurrence of <code>N</code> in the file will be replaced by <code>8</code> before compiling. Using <code>N</code> everywhere means you change the size in one place and the rest updates automatically.
<pre>int a[N] = {34, 17, 88, 5, 62, 41, 99, 23};</pre>	Declares an array of 8 integers and fills it with specific values at the same time. <code>a[0]</code> is 34, <code>a[1]</code> is 17, and so on.
<pre>int sum = 0, min = a[0], max = a[0];</pre>	Start sum at 0 (nothing accumulated yet). Start both min and max at <code>a[0]</code> , the first element. This is a standard pattern: assume the first element is both smallest and largest until proven otherwise.
<pre>for (int i = 0; i < N; i++)</pre>	Loop from index 0 to index 7 (N-1). We use <code>i < N</code> , not <code>i <= N</code> , because the last valid index is N-1.
<pre> sum += a[i];</pre>	Add the current element to the running total. After the loop, <code>sum</code> holds the total of all 8 values.
<pre> if (a[i] < min) min = a[i];</pre>	If this element is smaller than the current minimum, update <code>min</code> . After the loop, <code>min</code> holds the smallest value seen.
<pre> if (a[i] > max) max = a[i];</pre>	Same logic for maximum.
<pre>printf("Avg: %.2f\n", (double)sum / N);</pre>	Cast <code>sum</code> to <code>double</code> before dividing. Without the cast, <code>sum / N</code> would be integer division and drop the decimal part.

Expected output:

```
Sum: 369
Avg: 46.13
Min: 5
Max: 99
```

Example 2: Reversing an array

```
/* Swap a[i] and a[n-1-i] for i = 0 to n/2-1 */
void reverse(int a[], int n) {
    for (int i = 0; i < n / 2; i++) {
        int tmp = a[i];
        a[i] = a[n - 1 - i];
        a[n - 1 - i] = tmp;
    }
}
```

Guided In-Lab Exercises

Exercise 1: Read and summarize (Period B)

Write a program that:

1. Reads N (max 20) integers from the user.
2. Prints sum, average, min, max.
3. Counts how many values are above the average.

File: `lab11_stats.c`

Exercise 2: Linear search (Period B and C)

Write a function `int search(int a[], int n, int target)`. Read an array of 10 integers, then repeatedly prompt "Search for: " until the user enters -1. Print the index if found or "Not found".

File: `lab11_search.c`

Exercise 3: Rotate left (Period C)

Write a function `void rotate_left(int a[], int n)` that shifts each element one position to the left, wrapping the first element to the end. For example: `{1, 2, 3, 4, 5}` becomes `{2, 3, 4, 5, 1}`.

File: `lab11_rotate.c`

Challenge Problem

Write a function that checks whether an array is a **palindrome** (reads the same forwards and backwards). Return 1 if yes, 0 if no.

Common Pitfalls

Mistake	Symptom	Fix
Off-by-one: <code>i <= n</code> instead of <code>i < n</code>	Read/write past end of array (crash or garbage)	Use <code>i < n</code> ; array indices are 0 to n-1
Accessing uninitialized array	Garbage values	Always initialize or fill with <code>scanf</code> /assignment before reading
Forgetting to pass size to a function	Wrong results; potential out-of-bounds	Pass <code>n</code> as a separate parameter
<code>a = b</code> to copy arrays	Compilation error	Copy element by element with a loop

Submission and Rubric

Deliverable	Filename	Points
Read and summarize	<code>lab11_stats.c</code>	4
Linear search	<code>lab11_search.c</code>	3
Rotate left	<code>lab11_rotate.c</code>	3

Total: 10 points

Project assignment: The take-home individual project will be described in class this week. It is due at the **end of Week 14**. Details and tiered options in [Lab 14](#).

Further Reading

- King, Ch. 8 "Arrays"
- King, Ch. 13 "Strings" (first look; full coverage in Lab 12)
- Bubble sort and faster sorting algorithms / Big-O: see the [Data Structures & Algorithms references](#)
- K&R, Ch. 5 "Pointers and Arrays" §5.1 to §5.3

Lab 12: Arrays II: 2-D Arrays & Strings

Week	12
Duration	3 × 50 min (150 min)
Method	Lecture & Lab
Prerequisites	Lab 11 (1-D arrays, functions)

Why this lab matters: Tables of data are everywhere: a classroom grade sheet with students in rows and assignments in columns, a game board, or an image stored as a two-dimensional grid of pixels. Two-dimensional arrays let you represent and process any tabular structure. Strings are the way C programs handle text, from names and messages to file contents, and every program that interacts with human-readable data uses them. Together, 2-D arrays and strings cover the two most common structured data forms in systems programming.

Time allocation (one topic per period: tight week)

Period	Min	Activity
A	50	5 recap · 30 2-D array declaration and nested traversal · 15 matrix demo
B	50	5 transition · 30 strings, <code>\0</code> terminator, <code><string.h></code> basics · 15 string demo
C	50	35 independent exercises (matrix and text) · 10 challenge · 5 submit and Week 13 preview

Heavier `<string.h>` functions (`strtok`, `strstr`, `sprintf`) are take-home reading. Focus in class: the core concept of each period.

Learning Outcomes

By the end of this lab, you will be able to:

1. Declare and traverse a 2-D array using nested loops.
2. Compute row/column sums on a 2-D array.
3. Declare a C string as a `char` array and understand the `\0` null terminator.
4. Use `strlen`, `strcpy`, `strcmp`, and `strcat` from `<string.h>`.

Recap

In Week 11 you worked with 1-D arrays indexed by one number. A 2-D array adds a second index: rows and columns. Strings are also arrays: a C string is just an array of `char` values ending with the special value `\0` (null terminator).

Key Terms

In plain words: 2-D array A 2-D array is a grid (table) of values, all of the same type, with two indices: row and column. `int m[3][4]` creates a table with 3 rows and 4 columns, holding $3 \times 4 = 12$ integers. You access one cell with `m[row][col]`. Think of a spreadsheet: rows go top to bottom, columns go left to right. Always write the row index first, then the column index.

In plain words: C string In C, a string is not a built-in type; it is a `char` array with a special ending. The last character is always `'\0'` (null terminator, ASCII value 0), which marks “string ends here.” `char name[10] = "Alice"` stores 6 bytes: `'A'`, `'l'`, `'i'`, `'c'`, `'e'`, `'\0'`. Functions like `strlen` and `printf("%s", ...)` walk the array byte by byte and stop when they hit `'\0'`. If you forget to leave room for `'\0'`, these functions read past the array: undefined behavior.

In plain words: null terminator (`\0`) `'\0'` is the character with ASCII code 0. It is invisible when printed. Its only job is to mark the end of a string. Without it, C has no way to know where the string ends, because a `char` array has no built-in “length” field. The convention “the string ends at the first zero byte” is a design choice C made in the 1970s; it is simple but puts responsibility on the programmer to always include it.

Background: Period A: 2-D Arrays

2-D Array Row-Major Layout

	col 0	col 1	col 2	col 3
row 0	1	2	3	4
row 1	5	6	7	8
row 2	9	10	11	12

In memory (row-major):
1 2 3 4 | 5 6 7 8 | 9 10 11 12
row 0 row 1 row 2

Figure 12.1. 2-D array stored in row-major order.

```
int matrix[3][4];          /* 3 rows, 4 columns */
int m[2][3] = { {1,2,3}, {4,5,6} }; /* initialized */

m[0][0] /* row 0, col 0 -> 1 */
m[1][2] /* row 1, col 2 -> 6 */
```

Nested Traversal

```
int rows = 3, cols = 4;
for (int r = 0; r < rows; r++) {
    for (int c = 0; c < cols; c++) {
        printf("%4d", matrix[r][c]);
    }
    printf("\n");
}
```

Background: Period B: C Strings

String Layout in Memory

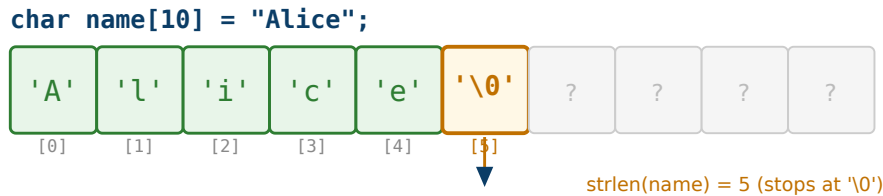


Figure 12.2. A C string and its null terminator.

```
char name[10] = "Alice";    /* stored: 'A','l','i','c','e','\0', garbage... */
char word[]   = "Hi";       /* size = 3: 'H','i','\0' */
```

Always allocate room for `\0`. A string of 5 characters needs at least 6 bytes.

Common `<string.h>` Functions

```
#include <string.h>

strlen(s)      /* number of chars before '\0' */
strcpy(dst, src) /* copy src into dst (dst must be large enough) */
strcat(dst, src) /* append src to dst */
strcmp(s1, s2)  /* 0 if equal; negative if s1 < s2; positive if s1 > s2 */
```

Under the Hood: chars are bytes

Each `char` is stored as one byte: its ASCII code. `'A'` = 65, `'a'` = 97, `'\0'` = 48, `'\0'` = 0. String functions work by walking the array byte by byte until they find 0. Missing the null terminator causes string functions to read past the array: a common bug.

Worked Examples

Example 1: Matrix row/column sums

```
/* lab12_matrix.c (outline) */
/* Reads a 3x3 matrix, prints it, then prints row sums and column sums */
#include <stdio.h>
#define R 3
#define C 3

int main(void) {
    int m[R][C];
    for (int r = 0; r < R; r++)
        for (int c = 0; c < C; c++)
            scanf("%d", &m[r][c]);
    for (int r = 0; r < R; r++) {
        int rsum = 0;
        for (int c = 0; c < C; c++) { printf("%4d", m[r][c]); rsum += m[r][c]; }
        printf(" | %d\n", rsum);
    }
    for (int c = 0; c < C; c++) {
        int csum = 0;
        for (int r = 0; r < R; r++) csum += m[r][c];
        printf("%4d", csum);
    }
    printf("\n");
    return 0;
}
```

Example 2: Palindrome check (string)

```
/* Returns 1 if s is a palindrome, 0 otherwise */
int is_palindrome(const char s[]) {
    int len = strlen(s);
    for (int i = 0; i < len / 2; i++) {
        if (s[i] != s[len - 1 - i]) return 0;
    }
    return 1;
}
```

Line by line: palindrome check

Line	What it does
<pre>int is_palindrome(const char s[])</pre>	Takes a string <code>s</code> as a <code>const char[]</code> . The <code>const</code> keyword promises: "I will not modify the string." The return type <code>int</code> means 1 = yes (palindrome), 0 = no.
<pre>int len = strlen(s);</pre>	Get the number of characters (not counting <code>'\0'</code>). <code>strlen</code> walks the array until it finds <code>'\0'</code> and counts the steps.
<pre>for (int i = 0; i < len / 2; i++)</pre>	Only need to check the first half. For a 5-character string, <code>len/2 = 2</code> so we check indices 0 and 1. The middle character (index 2) has no mirror.
<pre>if (s[i] != s[len - 1 - i]) return 0;</pre>	Compare the character at position <code>i</code> from the left with the character at position <code>i</code> from the right. If they differ, it is not a palindrome: return 0 immediately. For "level": compare <code>s[0]='l'</code> with <code>s[4]='l'</code> (match), <code>s[1]='e'</code> with <code>s[3]='e'</code> (match). Done.
<pre>return 1;</pre>	If the loop finished without finding a mismatch, every character matched its mirror: it is a palindrome. Return 1.

Examples:

```
"level" -> len=5, check i=0: 'l'=='l', i=1: 'e'=='e' -> return 1 (palindrome)  
"hello" -> len=5, check i=0: 'h' != 'o' -> return 0 (not palindrome)  
"racecar"-> len=7, check 3 pairs, all match -> return 1
```

Guided In-Lab Exercises

Exercise 1: Matrix operations (Period A to C)

Write a program that:

1. Reads a 3x4 integer matrix from the user.
2. Prints the matrix formatted with field width 5.
3. Prints the sum of each row.
4. Prints the sum of each column.
5. Prints the overall total.

File: `lab12_matrix.c`

Exercise 2: String statistics (Period B to C)

Write a program that reads a word (no spaces) and prints:

- Its length
- Whether it is a palindrome (yes/no)
- How many vowels it contains
- The word in uppercase (using `toupper` from `<ctype.h>`)

File: `lab12_text.c`

Exercise 3: Name sorter (Period C)

Read 5 names into a `char` array-of-strings (`char names[5][30]`). Sort them alphabetically using `strcmp` in a simple bubble sort. Print the sorted list.

File: `lab12_names.c`

Challenge Problem

Write a function `void transpose(int m[][N], int n)` that transposes a square matrix in-place (swaps element `[r][c]` with element `[c][r]`).

Common Pitfalls

Mistake	Symptom	Fix
Not leaving room for <code>\0</code>	String functions corrupt memory	Size = characters + 1
Comparing strings with <code>==</code>	Always compares pointer addresses, not content	Use <code>strcmp(s1, s2) == 0</code>
<code>strcpy</code> into a small buffer	Buffer overflow / crash	Ensure <code>dst</code> is large enough; use <code>strncpy</code> if unsure
Wrong column size in 2-D function param	Compiler error	The innermost dimension must be specified in the parameter

Submission and Rubric

Deliverable	Filename	Points
Matrix operations	lab12_matrix.c	5
String statistics	lab12_text.c	3
Name sorter	lab12_names.c	2

Total: 10 points

Further Reading

- King, Ch. 8 “Arrays”
- King, Ch. 13 “Strings”
- K&R, Ch. 5 “Pointers and Arrays”, Ch. 6 §6.4

Lab 13: Basic Data Structures

Week	13
Duration	3 × 50 min (150 min); tightest week, each period covering one concept
Method	Lecture & Lab
Prerequisites	Lab 12 (arrays, strings), Lab 09 to 10 (functions)

Why this lab matters: A student record has an ID, a name, and a GPA, which are three different types that belong together. A contact has a name, a phone number, and an email. Real data is almost always mixed: it has fields of different types that describe one thing. The `struct` is C's way of grouping those fields into a single named unit, similar to a row in a database table. Structs are the building block for every more complex data structure, such as linked lists and trees, that you will meet in Data Structures and beyond.

Time allocation (one topic per period)

Period	Min	Activity
A	50	5 recap · 30 <code>struct</code> definition and dot access · 15 struct demo
B	50	5 transition · 20 array-of-structs lab (guided) · 20 pointer intro (~20 min awareness demo) · 5 debrief
C	50	35 independent exercises (records program) · 10 challenge · 5 submit and Week 14 preview and project Q&A

"Combining arrays and structs" is the core in-class exercise. Deeper struct and pointer combination is take-home / project territory.

Learning Outcomes

By the end of this lab, you will be able to:

1. Define a `struct` and declare variables of that type.
2. Access struct members with the dot operator.
3. Create and traverse an array of structs.
4. Recognize that `&x` gives the memory address of `x`, connecting back to `scanf`.

Recap

In Weeks 11 to 12 you collected related values of the same type into arrays. But what if the data belongs together but has different types? A `struct` groups fields of *different* types under one name, like a row in a table.

Key Terms

In plain words: struct A `struct` (short for structure) is a way to group related variables of *different* types under one name. Think of a paper form with labeled fields: a student registration form has a field for ID number (integer), a field for name (text), and a field for GPA (decimal). All three pieces of data belong to the same person, so they should travel together. A `struct` is the C equivalent of that form.

Without structs, you would need three separate arrays (`int ids[100]` , `char names[100][30]` , `double gpas[100]`) and have to keep them in sync manually. With a struct, `Student roster[100]` keeps all three fields together per student.

In plain words: field (member) A *field* (also called a *member*) is one named slot inside a struct, just like one labeled box on a form. You access it with the dot operator: `s.gpa` reads the `gpa` field of the struct variable `s`. If you have a pointer to a struct, use the arrow operator: `p->gpa` is shorthand for `(*p).gpa`.

In plain words: pointer (awareness summary) A *pointer* is a variable that stores a memory address rather than a data value. `int *p = &x;` means “p holds the address of x.” You have been using `&` in `scanf(&score)` since Week 3; `&score` passes the address of `score` so `scanf` can write the value into it. `*p` (dereference) reads or writes the value *at* the address. Pointers are the gateway to dynamic memory and linked lists in later courses.

Background: Period A: Structures

Struct Record Layout

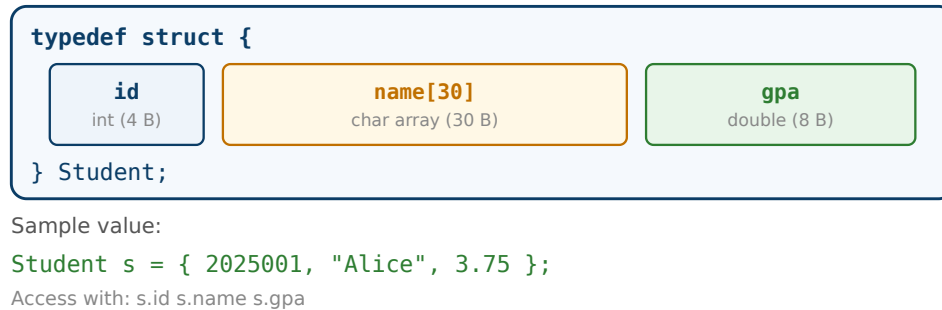


Figure 13.1. Layout of a struct record.

```
typedef struct {  
    int    id;  
    char   name[30];  
    double gpa;  
} Student;
```

```
Student s1 = {2025001, "Alice", 3.75};  
printf("%d  %s  %.2f\n", s1.id, s1.name, s1.gpa);
```

Background: Period B: Pointer Awareness (~20 min)

This is an **awareness demo only**: not assessed, not on the exam. The goal is to demystify `&` and prepare you for System Programming.

Pointer Arrow Diagram

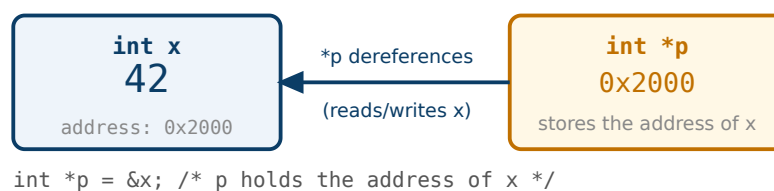


Figure 13.2. A pointer holds the address of a variable.

`&x` is the **address** of `x`, where it lives in memory. You have been using `&` with `scanf` since Week 3; now you know what it means.

```
int x = 42;
int *p = &x;      /* p points to x */
printf("%d\n", *p); /* dereference: read value at address p: prints 42 */
*p = 99;          /* modify x through p */
printf("%d\n", x); /* prints 99 */
```

This is the doorway to System Programming and Data Structures. In those courses you will use pointers to build linked lists, manage heap memory, and interact with the operating system. For now, just recognize the pattern.

Background: Period B continued: Array of Structs

```
Student roster[3] = {
    {2025001, "Alice", 3.75},
    {2025002, "Bob", 3.20},
    {2025003, "Carol", 3.90}
};

for (int i = 0; i < 3; i++) {
    printf("%-10s GPA: %.2f\n", roster[i].name, roster[i].gpa);
}
```

Worked Example: Student records

```
/* lab13_records.c (outline) */
#include <stdio.h>
#include <string.h>

#define MAX 5

typedef struct {
    int    id;
    char   name[30];
    double score;
} Student;

void print_roster(Student r[], int n);
Student find_top(Student r[], int n);

int main(void) {
    Student roster[MAX];
    for (int i = 0; i < MAX; i++) {
        printf("Student %d ID: ", i + 1);
        scanf("%d", &roster[i].id);
        printf("Name: ");
        scanf("%29s", roster[i].name);
        printf("Score: ");
        scanf("%lf", &roster[i].score);
    }
    print_roster(roster, MAX);
    Student top = find_top(roster, MAX);
    printf("Top student: %s (%.2f)\n", top.name, top.score);
    return 0;
}

void print_roster(Student r[], int n) {
    printf("%-8s %-12s %6s\n", "ID", "Name", "Score");
    for (int i = 0; i < n; i++)
        printf("%-8d %-12s %6.2f\n", r[i].id, r[i].name, r[i].score);
}

Student find_top(Student r[], int n) {
    Student best = r[0];
    for (int i = 1; i < n; i++)
        if (r[i].score > best.score) best = r[i];
    return best;
}
```

Line by line

Line	What it does
<pre>typedef struct { ... } Student;</pre>	Defines a new type called <code>Student</code> . After this definition, <code>Student</code> can be used anywhere you would use <code>int</code> or <code>double</code> to declare a variable. <code>typedef</code> saves you from writing <code>struct Student</code> every time.
<pre>void print_roster(Student r[], int n);</pre>	Prototype: declares that the function exists. It takes an array of <code>Student</code> and a count, and returns nothing.
<pre>Student roster[MAX];</pre>	Creates an array of <code>MAX</code> <code>Student</code> records. Each element is a complete <code>Student</code> with all three fields.
<pre>scanf("%d", &roster[i].id);</pre>	Reads an integer and stores it in the <code>id</code> field of <code>roster[i]</code> . The <code>.id</code> selects the field; <code>&</code> gives <code>scanf</code> its address so it can write there.
<pre>scanf("%29s", roster[i].name);</pre>	Reads a word (up to 29 characters, leaving room for <code>'\0'</code>) into the <code>name</code> field. Note: <code>name</code> is a char array, so no <code>&</code> needed; the array name already acts as an address. The <code>29</code> prevents a buffer overflow.
<pre>scanf("%lf", &roster[i].score);</pre>	Reads a <code>double</code> . Must use <code>%lf</code> (lowercase L, not letter i) for <code>double</code> with <code>scanf</code> .
<pre>Student top = find_top(roster, MAX);</pre>	Calls <code>find_top</code> , which returns a <code>Student</code> value by copy. The whole struct is copied into <code>top</code> .
<pre>Student best = r[0];</pre>	Inside <code>find_top</code> : start by assuming the first student has the top score.
<pre>if (r[i].score > best.score) best = r[i];</pre>	If student <code>i</code> beats the current best, copy the entire <code>Student</code> struct into <code>best</code> . Structs can be assigned with <code>=</code> , unlike arrays.
<pre>return best;</pre>	Returns a copy of the best-scoring student record to the caller.

Sample interaction:

```
Student 1 ID: 2025001
Name: Alice
Score: 88.5
Student 2 ID: 2025002
Name: Bob
Score: 92.0
...
ID      Name      Score
2025001 Alice      88.50
2025002 Bob       92.00
...
Top student: Bob (92.00)
```

Guided In-Lab Exercises

Exercise 1: Contact book (Period A and B)

Define a `struct Contact` with name, phone (string), and email (string). Create an array of 3 contacts, read them from the user, then print a formatted table.

File: `lab13_contacts.c`

Exercise 2: Records program (Period C)

Implement the Student roster from the worked example. Add a function that computes the class average score.

File: `lab13_records.c`

Challenge Problem

Add a function `void sort_by_score(Student r[], int n)` that sorts the roster in descending order of score using bubble sort. Print the sorted roster.

Common Pitfalls

Mistake	Symptom	Fix
<code>s1 == s2</code> to compare structs	Compilation error	Compare field by field
Forgetting <code>&</code> for struct scalar fields in <code>scanf</code>	Crash	<code>scanf("%d", &s.id)</code>
Using <code>&</code> for char array fields in <code>scanf</code>	Unnecessary but harmless (or wrong)	<code>scanf("%29s", s.name)</code> with no <code>&</code> for array fields
<code>printf("%s\n", &s.name)</code>	Extra <code>&</code> is noisy	Use <code>s.name</code> directly

Submission and Rubric

Deliverable	Filename	Points
Contact book	<code>lab13_contacts.c</code>	4
Student records	<code>lab13_records.c</code>	6

Total: 10 points

Further Reading

- King, Ch. 16 "Structures, Unions, and Enumerations"
- K&R, Ch. 6 "Structures"
- [Bonus: Dynamic Memory](#)

Lab 14: Debugging, Analysis and Project Build

Week	14
Duration	3 × 50 min (150 min)
Method	Lecture & Lab + Project Submission
Prerequisites	Labs 01–13 (entire course)

Time allocation

Period	Min	Activity
A	50	5 recap · 30 systematic debugging (printf-tracing, rubber-duck, gdb intro) · 15 live demo
B	50	30 program analysis (read & trace unfamiliar code) · 20 project finalization
C	50	35 project finalization + submission · 10 oral defense preview + Q&A · 5 wrap-up

This is the **capstone** debugging session, not the first time. You have been reading error messages since Week 2 and meeting per-topic pitfalls all term. Today you assemble everything into a systematic approach.

Learning Outcomes

By the end of this lab, you will be able to:

1. Apply a systematic debugging process: reproduce → isolate → identify → fix → verify.
2. Use `printf`-tracing and the rubber-duck method to find a bug.
3. Use `gdb` to set a breakpoint, step through code, and inspect a variable.
4. Read and trace an unfamiliar C program and predict its output.
5. Submit your individual project (artifact + README).

Recap

Over the last 13 weeks you have encountered:

- Compile-time errors (Weeks 2, 3, 4: syntax, types, missing includes)
- Runtime crashes and wrong output (Weeks 6–7: loops, off-by-one, infinite loops)
- Logic bugs in conditionals and arrays (Weeks 5, 11)

Today you learn to attack any bug with a repeatable method.

Background: Systematic Debugging

The Process

1. **Reproduce:** confirm you can make the bug happen consistently.
2. **Isolate:** narrow down *where* in the code the problem occurs.
3. **Identify:** understand *why* it goes wrong.
4. **Fix:** make the smallest change that corrects the problem.
5. **Verify:** test with the original case *and* a few others.

printf-Tracing

The simplest debugger: add temporary `printf` statements to print variable values at key points. Check whether they match what you expect.

```
printf("DEBUG: i=%d, sum=%d\n", i, sum);
```

Remove or comment out `DEBUG:` lines before submitting.

Rubber-Duck Debugging

Explain your code, line by line, to an imaginary rubber duck (or a classmate). The act of explaining often surfaces the assumption you got wrong.

Intro to gdb: Three Commands

```
gcc -g myprogram.c -o myprogram    # compile with debug info
gdb ./myprogram                    # start debugger
```

Inside `gdb`:

```
(gdb) break main      # set breakpoint at main
(gdb) run              # start program; stops at breakpoint
(gdb) next             # execute one line (step over functions)
(gdb) print x          # print the value of variable x
(gdb) quit             # exit gdb
```

You only need these five commands for now. Full `gdb` is covered in System Programming.

Worked Example: Tracing an Unfamiliar Program

Given this program (do not run it yet):

```
#include <stdio.h>

int mystery(int n) {
    int result = 0;
    for (int i = 1; i <= n; i++) {
        if (i % 2 != 0) result += i;
    }
    return result;
}

int main(void) {
    printf("%d\n", mystery(7));
    return 0;
}
```

1. What does `mystery` compute?
2. What does `mystery(7)` return?
3. What is printed?

Trace it on paper first, then run and verify.

Guided In-Lab Exercises

Exercise 1: Bug hunt (Period A)

The instructor provides `buggy.c`, a program with 3 deliberate bugs (at least one compile-time, one logic bug). Your task:

1. Compile and read the errors.
2. Use `printf`-tracing to find the logic bug.
3. Fix all bugs. Write a comment near each fix: `/* BUG FIX: ... */`

File: submit the corrected `lab14_buggy.c`

Exercise 2: Trace the program (Period B)

Given `mystery.c` (provided), predict the output for inputs 5, 10, and 0. Write your predictions in a comment at the top, then run and compare.

File: `lab14_trace.c` (add your comment block)

Exercise 3: Project finalization and submission (Period B–C)

Use this time to:

- Run your project on several test inputs.
 - Fix any remaining bugs.
 - Write or complete your README (see requirements below).
 - Submit to LMS before the session ends.
-

Project Submission Requirements

The **individual take-home project** counts for **30% of the final grade** (artifact). The oral defense (30%) is in Week 15.

Project Options (choose one)

Option	Description
A: Contact Book	Store/search/display contacts; struct array; string operations
B: Grade Manager	Read student scores; compute stats; letter grades; formatted report
C: Number Guessing Game	Configurable range; track attempts; high-score persistence
D: Calculator Suite	Multi-function calculator with history; functions for each operation
Custom	Propose your own to the instructor before Week 13

Submission Checklist

- ☐ `main.c` (or multiple `.c` files with one `main` entry point)
- ☐ `README.txt` or `README.md` with: what the program does, how to compile and run it, known limitations, list of features implemented
- ☐ Compiles with `gcc *.c -o project -Wall -Wextra -std=c99` without errors
- ☐ Works for basic test cases

Artifact Rubric (30%)

Criterion	Weight
Compiles and runs	5%
Core features work correctly	10%
Code quality (functions, naming, comments, style)	8%
README / report	4%
Bonus features	3% (optional)

Common Pitfalls

Mistake	Fix
Submitting code that does not compile	Compiler errors must be fixed first
No README	Write at minimum: “what it does” and “how to compile/run”
Only fixing the specific test case but not the root cause	Apply the systematic process: fix the underlying bug
Leaving <code>DEBUG: printf</code> lines in submitted code	Remove or comment them before submission

Submission and Rubric

Deliverable	Filename	Points
Corrected buggy program	<code>lab14_buggy.c</code>	4
Trace exercise with predictions	<code>lab14_trace.c</code>	2
Project artifact + README	<code>project/</code> folder	30% of final grade

Weekly lab deliverables: **6 points** (graded with the assignments 10% weight).

Further Reading

- [Appendix: Debugging Tips](#): full error catalog + gdb quick reference
- K&R, Ch. 2–4 (for tracing unfamiliar programs)
- [Bonus: File I/O](#): if your project needs to save data to disk

Lab 15: Final: Project Demo + Individual Oral Defense

Week	15
Duration	150 min (single continuous block)
Method	Examination: demo and individual oral defense
Scope	Entire course; assessed through the project (emphasizes Weeks 9–14)
Weight	30% of final grade

Time allocation

Phase	Min	Activity
Setup	5	Confirm project runs on the lab machine
Student rotations	~135	~10–12 min per student: demo · Q&A · live modification
Buffer	10	Technical issues; parallel stations if class is large

What This Exam Is

The final is **not** a written test or a timed coding sprint from scratch. It is an **individual oral defense**: you present your project and answer questions about it live. You demonstrate that you wrote it, understand it, and can extend it on the spot.

Each student gets approximately **10–12 minutes**:

1. **Live demo (2–3 min)**: run your program and show it works for a sample scenario.
 2. **Explain-your-code Q&A (5–6 min)**: the examiner asks about specific parts of your code. Typical questions: “What does this function do?”, “Why did you use a struct here?”, “Walk me through what happens when the user enters X.”
 3. **Live modification (3–4 min)**: the examiner asks you to add or change a small feature *during the session*. This validates authorship and measures understanding beyond memorization.
-

What Is Being Assessed

The oral defense assesses Weeks 9–14 *through your project*:

Topic	How it shows up
Functions (Wk 9–10)	Can you explain what each function does? Why split the code this way?
Arrays (Wk 11–12)	Can you trace array indexing? Explain the loop bounds?
Strings (Wk 12)	If your project processes strings, can you explain the <code>\0</code> terminator?
Structs (Wk 13)	Can you explain why you grouped data into a struct?
Debugging (Wk 14)	Can you find and fix a bug in your own code in real time?

Oral Defense Rubric (30%)

Criterion	Description	Weight
Runs & meets spec	Program runs without crashes; core features work as described in README	8%
Code understanding	Student can explain any part of their code accurately	10%
Live modification	Student successfully implements the requested change (partial credit for correct approach)	8%
Presentation clarity	Clear communication; organized demo	4%

Preparing for the Oral Defense

Know Your Own Code

- Read every line of your project before the oral defense.
- Be ready to explain: what each function does, why you chose that data structure, how your loop terminates, what happens on invalid input.
- If you copied a snippet from an example, understand it. You will be asked about it.

Practice Typical Questions

- “What does `main` do first?”
- “Why did you use a `while` loop here instead of `for`?”
- “What happens if the user enters a negative number?”
- “How does your struct store the data?”
- “Walk me through the output for this input: ...”

Prepare for the Live Modification

Past modifications have included:

- “Add a feature that prints the total number of items.”
- “Make the program reject negative input.”
- “Add a function that returns the average of your array.”

These are deliberate small changes in scope. You are not expected to redesign the program. Writing clean, modular code in Week 14 makes Week 15 easier.

Oral Defense Scoring Guide

Score band	Meaning
90–100%	Explains all code fluently; live modification complete and correct
75–89%	Explains most code with minor gaps; modification mostly correct
60–74%	Explains core features; modification partial
Below 60%	Cannot explain key parts; modification not attempted

Logistics

- Bring your project files on the lab machine (or a USB drive as backup).
 - Have a terminal open with the project directory ready.
 - Compile your project fresh at the start of the session: `gcc *.c -o project -Wall -std=c99`
 - If two or more lab machines are available, the class splits into parallel stations so wait time is minimal.
-

After the Exam

This is the last graded activity of the semester. Final grades are composed of:

Component	Weight
Midterm (Week 8)	20%
Project artifact (submitted Week 14)	30%
Final: demo + oral defense (today)	30%
Attendance	10%
Assignments (Weeks 2–13)	10%

Congratulations

You have completed **Computer Programming I**. You can now write, compile, debug, and explain C programs that use variables, control flow, functions, arrays, strings, and basic data structures.

Where to go next:

- **Computer Programming II / System Programming:** pointers in depth, memory management, file I/O, processes
- **Data Structures:** linked lists, stacks, queues, trees, sorting algorithms
- **Computer Architecture:** what the CPU and memory really do
- **Operating Systems:** how programs interact with the OS kernel

The [References appendix](#) lists recommended resources for each path.

Further Reading

- [Grading Rubrics](#): full scoring criteria
- [Debugging Tips](#): in case you need to fix a bug during the oral defense

C Style Guide

Consistent code style makes programs easier to read, debug, and grade. Follow these conventions in all submitted work.

File Header Block

Every `.c` file must begin with a comment block:

```
/*
 * Course   : 400521-004 Computer Programming I
 * Instructor: Yushintia Pramitarini
 * Name      : [Your Name]
 * Student ID: [Your ID]
 * Date      : [YYYY-MM-DD]
 * Lab       : [Lab 0X: Topic Name]
 * Filename  : [filename.c]
 * Description: [one sentence about what this program does]
 */
```

Naming Conventions

Element	Style	Example
Variables	snake_case	total_score, num_students
Constants (#define , const)	SCREAMING_SNAKE	MAX_SIZE, PI
Functions	snake_case	compute_average(), print_menu()
Types (typedef struct)	PascalCase	Student, ContactEntry
Loop counters	short is fine	i, j, r, c

Use **descriptive names**: `score` is better than `x`; `num_items` is better than `n` unless `n` is conventional in a mathematical context.

Indentation and Spacing

- Use **4 spaces** per indentation level (or tabs set to 4).
- Do **not** mix spaces and tabs.
- Opening brace `{` on the **same line** as the control statement.
- Closing brace `}` on its **own line**.

```
/* Good */
if (score >= 90) {
    printf("A\n");
} else {
    printf("Below A\n");
}

/* Bad: no braces, inconsistent spacing */
if(score>=90) printf("A\n");
else printf("Below A\n");
```

Always use braces `{ }` even for single-statement bodies.

Comments

- Write a comment for every function explaining what it does, its parameters, and its return value.
- Inside functions, comment non-obvious logic (not every line).
- Do not comment what the code obviously says (`i++`; `/* increment i */` is noise).

```
/*
 * compute_average: returns the average of the first n elements of array a.
 * Parameters: a - integer array, n - number of elements (n > 0)
 * Returns: average as a double
 */
double compute_average(int a[], int n) {
    int sum = 0;
    for (int i = 0; i < n; i++) sum += a[i];
    return (double)sum / n;
}
```

Functions

- **One function = one job.** If a function does two things, split it.
- Keep `main` short; call functions for each major task.

- Declare prototypes at the top of the file.
 - Avoid global variables; pass data as parameters and return results.
-

Constants

```
/* Good */
#define MAX_STUDENTS 30
const double TAX_RATE = 0.10;

/* Bad: magic numbers */
if (count > 30) ...      /* what is 30? */
total = price * 1.10;    /* why 1.10? */
```

Name every non-obvious constant.

Submission Filename Conventions

Lab	Filename
Lab 02 card	lab02_card.c
Lab 06 guess	lab06_guess.c
Project	project/main.c + project/README.txt

Use **lowercase with underscores**. Do not use spaces in filenames.

Quick Checklist Before Submission

- ☐ File header block present and filled in
- ☐ Compiles with `gcc -Wall -Wextra -std=c99` without errors or warnings
- ☐ No debug `printf` statements left in
- ☐ Descriptive variable and function names
- ☐ Consistent 4-space indentation
- ☐ All functions have a comment explaining their purpose
- ☐ Correct filename format

Grading Rubrics

This appendix documents the scoring criteria for all three graded categories: weekly assignments, the midterm, and the final.

1. Weekly Assignment Rubric (Assignments 10%)

Each lab deliverable is scored out of 10 points.

Criterion	Points	Description
Compiles	2	Program compiles with <code>gcc -Wall -Wextra -std=c99</code> without errors
Correctness	4	Output matches the expected result for the provided test cases
Style & readability	2	Follows the Style Guide : header block, names, indentation, comments
Approach	2	Algorithm is appropriate; no unnecessary complexity; code is concise
Total	10	

If the program does not compile, **Correctness** and **Approach** are not awarded. Fix all compiler errors before submitting.

2. Midterm Rubric: Live In-Lab Coding (Midterm 20%)

The midterm consists of **3–5 short tasks** (Week 8, covers Weeks 1–7). Each task is scored independently; partial credit is available per task.

Per-Task Rubric (10 points per task)

Criterion	Points	Description
Compiles without errors	2	Program must compile
Correct output: sample test	4	Produces correct output for the provided test case
Correct output: hidden test	2	Produces correct output for an unseen test case
Code style	1	Clean indentation, meaningful names
Algorithm	1	Appropriate approach; no unnecessary code

Final Midterm Score Calculation

$$\text{Midterm score} = (\text{sum of task scores} / \text{total available task points}) \times 20$$

Example: if there are 4 tasks (40 points total) and a student earns 32 points, their midterm contribution is $(32/40) \times 20 = 16\%$ of the final grade.

3. Project Artifact Rubric (Project 30%)

The artifact is the submitted code + README, **due at the end of Week 14**.

Criterion	Weight	Description
Compiles and runs	5%	<code>gcc *.c -o project -Wall -std=c99</code> succeeds; program starts and does not immediately crash
Core features work	10%	All features described in the README behave correctly for valid input
Code quality	8%	Functions used appropriately; naming, indentation, comments meet the style guide
README / report	4%	Explains what the program does, how to compile and run, known limitations
Bonus features	3%	Optional: additional features beyond the core requirement
Total	30%	

4. Final Exam Rubric: Project Demo + Oral Defense (Final 30%)

The final is an **individual oral defense** of the submitted project (Week 15). Each student has approximately 10–12 minutes.

Criterion	Weight	Description
Runs & meets spec	8%	Program runs without crashes during the demo; core features demonstrated
Code understanding	10%	Student explains any code section accurately when asked; no major misconceptions
Live modification	8%	Student successfully implements the examiner's small requested change (partial credit for correct direction)
Presentation clarity	4%	Demo is organized; student communicates clearly
Total	30%	

Oral Defense Scoring Guide

Score band	Code understanding	Live modification
Full marks	Explains all parts accurately; can reason about edge cases	Completes the modification correctly
75%	Explains most parts; minor gaps	Correct approach; minor syntax error
50%	Explains some parts; several gaps	Correct direction but incomplete
25%	Struggles with most of own code	Unable to start the modification
0%	Cannot explain any part	No attempt

5. Grade Composition Summary

Component	Weight	When
Midterm: live coding	20%	Week 8
Project: artifact	30%	Submitted Week 14
Final: demo + oral defense	30%	Week 15
Attendance	10%	Ongoing
Assignments	10%	Weeks 2–13
Total	100%	

Grade distribution (per university policy): $A \leq 40\%$ · $B \leq 40\%$ · $C \leq 20\%$ of enrolled students.

Debugging Tips & Common Errors

This appendix is your reference for the two categories of bugs you will encounter: **compile-time errors** (the program won't build) and **runtime bugs** (the program builds but misbehaves).

Part 1: Reading Compiler Error Messages

Anatomy of a GCC Error

```
hello.c:5:18: error: expected ';' before 'return'
```

Part	Meaning
hello.c	Which file
5	Which line
18	Which column (character position)
error:	Fatal: program won't compile
warning:	Non-fatal but dangerous; treat as errors
Message text	What went wrong

Rule: Fix the **first** error first. Later errors are often caused by the first one. After each fix, recompile before fixing the next.

Part 2: Common Compile-Time Errors

Error message	Likely cause	Fix
expected ';' before ...	Missing semicolon	Add ; at the end of the previous statement
implicit declaration of function 'printf'	Missing #include <stdio.h>	Add the include
undeclared identifier 'x'	Variable used before declared	Declare int x; before first use
too few arguments to function	Wrong number of arguments in a call	Check the function prototype
expected expression	Missing value where one is needed	Check for typos, missing operand
lvalue required as left operand of assignment	5 = x; (cannot assign to a literal)	Flip to x = 5;
incompatible types	Assigning wrong type	Check that types match, or cast explicitly
redefinition of ...	Variable or function declared twice	Remove the duplicate

Part 3: Common Logic (Runtime) Bugs

These compile fine but produce wrong results or crashes.

Off-by-one Errors

```
int a[5];
for (int i = 0; i <= 5; i++) a[i] = 0; /* Bug: i=5 accesses a[5] - out of bounds! */
for (int i = 0; i < 5; i++) a[i] = 0; /* Correct */
```

How to find: trace the first and last iterations manually.

Infinite Loop

```
int i = 0;
while (i < 10) {
    printf("%d\n", i);
    /* forgot i++ */
}
```

How to find: add `printf("DEBUG i=%d\n", i);` inside the loop. If the same value prints forever, you have a missing update.

Using = Instead of ==

```
if (x = 5) { ... } /* Bug: assigns 5 to x, condition is always true */
if (x == 5) { ... } /* Correct */
```

Compile with `-Wall`; GCC warns about this pattern.

Integer Division Giving 0

```
double rate = 1 / 3; /* Bug: integer division → 0; then stored as 0.0 */
double rate = 1.0 / 3; /* Correct: 0.333... */
```

Forgetting & in scanf

```
int x;
scanf("%d", x); /* Bug: passes the value of x, not its address */
scanf("%d", &x); /* Correct */
```

Often causes a segmentation fault.

Not Null-Terminating a String

```
char s[3] = {'H', 'i'}; /* s[2] is garbage, not '\0' */
printf("%s\n", s); /* undefined: reads past end */

char s[3] = {'H', 'i', '\0'}; /* Correct */
/* or simply: */
char s[] = "Hi";
```

Uninitialized Variable

```
int total;
printf("%d\n", total); /* garbage: value is undefined */

int total = 0;          /* always initialize */
```

Part 4: printf-Tracing Recipe

When a program compiles but gives wrong results:

1. **Identify the suspected region:** narrow down which function or loop has the bug.
2. **Add `DEBUG: printf` statements** before and after key operations:

```
printf("DEBUG: entering loop, i=%d, sum=%d\n", i, sum);
```

3. **Run and read the output:** find where values deviate from what you expect.
 4. **Fix the root cause**, not just the symptom.
 5. **Remove all `DEBUG: lines`** before submitting.
-

Part 5: gdb Quick Reference

Compile with debug info: `gcc -g file.c -o prog`

Command	Effect
<code>gdb ./prog</code>	Start debugger
<code>break main</code>	Set breakpoint at the start of <code>main</code>
<code>break 42</code>	Set breakpoint at line 42
<code>run</code>	Start program; stops at first breakpoint
<code>next</code>	Execute next line (step over function calls)
<code>step</code>	Execute next line (step into function calls)
<code>print x</code>	Print value of variable <code>x</code>
<code>print *p</code>	Dereference pointer <code>p</code>
<code>continue</code>	Run until next breakpoint
<code>quit</code>	Exit gdb

Part 6: Debugging Checklist

Before asking for help, go through this checklist:

- ☐ Compile with `-Wall -Wextra`: have you read all warnings?
- ☐ Is the first error fixed first?
- ☐ Did you add `printf`-tracing to narrow down the bad region?
- ☐ Did you trace the loop (first and last iteration) on paper?
- ☐ Did you check `=` vs `==` in every condition?
- ☐ Are all arrays indexed from 0 to $n-1$?
- ☐ Is every `scanf` call using `&`?
- ☐ Are all variables initialized before use?
- ☐ Are all strings null-terminated?

References & Further Reading

Primary Textbooks

1. **K. N. King**, *C Programming: A Modern Approach*, 2nd ed., W. W. Norton & Company. The primary textbook for this course. Beginner-friendly, comprehensive, with extensive exercises.
2. **Brian W. Kernighan & Dennis M. Ritchie**, *The C Programming Language*, 2nd ed., Prentice Hall. The canonical reference written by C's creators. Concise and authoritative; used for chapter mapping below.

Supplementary Textbooks

3. **Stephen Prata**, *C Primer Plus*, 6th ed., Addison-Wesley. Very gentle introduction; ideal for absolute beginners who want more worked examples and explanation.
 4. **Paul Deitel & Harvey Deitel**, *C How to Program*, 8th ed., Pearson. Extensive examples and exercises; strong on I/O and structured programming.
-

Week-by-Week Chapter Map

Week	Topic	King (K)	K&R
1	Introduction to programming	K Ch. 1	Ch. 1
2	Program structure, printf	K Ch. 2	Ch. 1
3	Variables, types, scanf	K Ch. 3-4, 7	Ch. 1-2
4	I/O formatting, operators	K Ch. 3-4	Ch. 2
5	Conditionals	K Ch. 5	Ch. 3
6	while / do-while loops	K Ch. 6	Ch. 3
7	for / nested loops	K Ch. 6	Ch. 3
8	Midterm	n/a	n/a
9	Functions I	K Ch. 9	Ch. 4
10	Functions II, scope, recursion	K Ch. 9	Ch. 4
11	1-D Arrays	K Ch. 8	Ch. 5.1-5.3
12	2-D Arrays & Strings	K Ch. 8, 13	Ch. 5-6
13	Structs, pointer intro	K Ch. 16	Ch. 6
14	Debugging, project	n/a	n/a
15	Final	n/a	n/a

Online Resources

Resource	URL	Use
cppreference.com	https://en.cppreference.com/w/c	Standard library reference
OnlineGDB	https://www.onlinegdb.com/online_c_compiler	Browser C compiler (no install)
Programiz online C	https://www.programiz.com/c-programming/online-compiler/	Alternative browser compiler
GNU GDB manual	https://sourceware.org/gdb/current/onlinedocs/gdb/	Full gdb reference

Where to Go Next

After completing this course, you have several paths forward:

System Programming

Topics you will study: pointers in depth, dynamic memory (`malloc` / `free`), file I/O, processes and threads, system calls, the Unix API. **Recommended reading:** K&R Chapters 5–7; *The Linux Programming Interface* (Kerrisk). **Preview in this course:** [Bonus: Dynamic Memory](#), [Bonus: File I/O](#).

Data Structures & Algorithms

Topics you will study: linked lists, stacks, queues, binary trees, hash tables, sorting and searching algorithms, complexity analysis (Big-O). **Recommended reading:** *Data Structures in C* (Tenenbaum); *Introduction to Algorithms* (CLRS). **Preview in this course:** the pointer intro in Lab 13.

Computer Architecture

Topics you will study: the CPU instruction set, memory hierarchy (cache, RAM, disk), pipelining, assembly language. **Connection to this course:** every “Under the Hood” box in the lab pages.

Operating Systems

Topics you will study: process management, scheduling, virtual memory, file systems, I/O.

Prerequisites: System Programming, Computer Architecture.

Bonus: Bitwise Operators

This is optional enrichment, beyond the CS1 curriculum. Bitwise operators are not assessed in this course. They are introduced here for students who are curious, and because they appear in Week 4's optional demo and in System Programming.

What Are Bitwise Operators?

All data in a computer is stored as **bits** (0s and 1s). Bitwise operators work directly on the individual bits of an integer value.

Operator	Name	Example
<code>&</code>	AND	<code>5 & 3 → 1</code>
<code> </code>	OR	<code>5 3 → 7</code>
<code>^</code>	XOR	<code>5 ^ 3 → 6</code>
<code>~</code>	NOT (complement)	<code>~5 → -6</code>
<code><<</code>	Left shift	<code>5 << 1 → 10</code>
<code>>></code>	Right shift	<code>5 >> 1 → 2</code>

Bit-Level Arithmetic

5 in binary: 0000 0101

3 in binary: 0000 0011

5 & 3: 0000 0001 = 1 (AND: 1 only where both are 1)

5 | 3: 0000 0111 = 7 (OR: 1 where either is 1)

5 ^ 3: 0000 0110 = 6 (XOR: 1 where exactly one is 1)

~5: 1111 1010 = -6 (flip all bits; two's complement)

5 << 1: 0000 1010 = 10 (shift left = multiply by 2)

5 >> 1: 0000 0010 = 2 (shift right = divide by 2, integer)

Practical Uses

Check if a number is even or odd

```
if (n & 1) printf("odd\n");  
else      printf("even\n");
```

`n & 1` isolates the last bit; if it is 1, the number is odd.

Toggle a bit (flip bit k)

```
int mask = 1 << k; /* bit k is 1, rest are 0 */  
x = x ^ mask;      /* flip bit k of x */
```

Set a bit

```
x = x | (1 << k); /* force bit k to 1 */
```

Clear a bit

```
x = x & ~(1 << k); /* force bit k to 0 */
```

Where You Will See This Again

- **System Programming:** flags passed to `open()`, `chmod()`, `ioctl()` are bit fields.
 - **Embedded systems:** controlling hardware registers at the bit level.
 - **Network programming:** IP masks, protocol flags.
 - **Cryptography:** XOR is a fundamental building block.
-

A Small Demo Program

```
/* bitwise_demo.c */
#include <stdio.h>

void print_bits(unsigned int x) {
    for (int i = 7; i >= 0; i--)
        printf("%d", (x >> i) & 1);
    printf("\n");
}

int main(void) {
    unsigned int a = 5, b = 3;
    printf("a      = "); print_bits(a);
    printf("b      = "); print_bits(b);
    printf("a & b   = "); print_bits(a & b);
    printf("a | b   = "); print_bits(a | b);
    printf("a ^ b   = "); print_bits(a ^ b);
    printf("a << 1 = "); print_bits(a << 1);
    return 0;
}
```

Bonus: File I/O

This is optional enrichment, beyond the CS1 curriculum. File I/O is not assessed in this course. It is introduced here for students who want their projects to save data between runs. You will study file I/O thoroughly in **System Programming**.

Why File I/O?

All programs so far read from the keyboard and print to the screen. When the program exits, all data is lost. File I/O lets you **persist data**: save it to a file and read it back later.

Opening and Closing Files

```
#include <stdio.h>

FILE *fp;
fp = fopen("data.txt", "w"); /* open for writing (creates or overwrites) */
if (fp == NULL) {
    printf("Error: cannot open file.\n");
    return 1;
}
/* ... use fp ... */
fclose(fp);
```

Mode	Meaning
"r"	Open for reading (file must exist)
"w"	Open for writing (creates or overwrites)
"a"	Open for appending (adds to end)
"r+"	Open for reading and writing

Always check that `fopen` did not return `NULL`: the file might not exist or might not be writable. **Always call `fclose`**: this flushes buffered data to disk.

Writing to a File

```
FILE *fp = fopen("output.txt", "w");
if (fp == NULL) { perror("fopen"); return 1; }

fprintf(fp, "Name: %s\n", "Alice");      /* like printf, but to a file */
fprintf(fp, "Score: %d\n", 95);
fputs("Goodbye!\n", fp);                 /* write a string */

fclose(fp);
```

Reading from a File

```
FILE *fp = fopen("data.txt", "r");
if (fp == NULL) { perror("fopen"); return 1; }

char line[100];
while (fgets(line, sizeof(line), fp) != NULL) { /* read one line */
    printf("%s", line);                          /* line includes '\n' */
}

fclose(fp);
```

Or read formatted values:

```
int id;
char name[30];
double score;
while (fscanf(fp, "%d %29s %lf", &id, name, &score) == 3) {
    printf("%-8d %-12s %.2f\n", id, name, score);
}
```

A Complete Example: Save and Load a Contact List

```
/* contacts_file.c (outline) */
#include <stdio.h>
#include <string.h>

typedef struct { char name[30]; char phone[15]; } Contact;

void save(Contact c[], int n, const char *filename) {
    FILE *fp = fopen(filename, "w");
    if (!fp) { perror("save"); return; }
    for (int i = 0; i < n; i++)
        fprintf(fp, "%s %s\n", c[i].name, c[i].phone);
    fclose(fp);
}

int load(Contact c[], int max, const char *filename) {
    FILE *fp = fopen(filename, "r");
    if (!fp) return 0; /* file does not exist yet */
    int n = 0;
    while (n < max && fscanf(fp, "%29s %14s", c[n].name, c[n].phone) == 2)
        n++;
    fclose(fp);
    return n;
}
```

Where You Will See This Again

- **System Programming:** `open()`, `read()`, `write()` (low-level POSIX file API).
- **Databases:** file-backed storage, indexing, transactions.
- **Any real project:** configuration files, logs, data persistence.

Bonus: Dynamic Memory (malloc / free)

This is optional enrichment, beyond the CS1 curriculum. Dynamic memory allocation is not assessed in this course. It is the first topic of **System Programming** and **Data Structures**. This page is a teaser.

The Problem with Fixed-Size Arrays

All arrays in this course have a size fixed at **compile time**:

```
int scores[100]; /* must decide at compile time; what if there are 200 students? */
```

What if you do not know the size until the program runs? Dynamic memory allocation solves this.

Stack vs Heap

Memory region	What lives there	Lifetime
Stack	Local variables, function call frames	Freed automatically when function returns
Heap	Memory you allocate with <code>malloc</code>	Lives until you call <code>free</code>

Local arrays are on the stack (limited size, typically 1–8 MB). Dynamic arrays are on the heap (limited only by available RAM).

malloc and free

```
#include <stdlib.h>

/* Allocate an array of n ints */
int *scores = malloc(n * sizeof(int));
if (scores == NULL) {
    printf("Out of memory!\n");
    return 1;
}

/* Use it exactly like a regular array */
for (int i = 0; i < n; i++) scores[i] = 0;

/* Free when done - required! */
free(scores);
scores = NULL; /* prevent accidental use after free */
```

`malloc(size)` returns a pointer to the newly allocated block, or `NULL` on failure. `free(ptr)` returns the memory to the system.

A Concrete Example: Read N scores at runtime

```
/* dynamic_scores.c */
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n;
    printf("How many scores? ");
    scanf("%d", &n);

    int *scores = malloc(n * sizeof(int));
    if (!scores) { fprintf(stderr, "malloc failed\n"); return 1; }

    for (int i = 0; i < n; i++) {
        printf("Score %d: ", i + 1);
        scanf("%d", &scores[i]);
    }

    int sum = 0;
    for (int i = 0; i < n; i++) sum += scores[i];
    printf("Average: %.2f\n", (double)sum / n);

    free(scores);
    return 0;
}
```

Common Dynamic Memory Bugs

Bug	Description
Memory leak	<code>malloc</code> without <code>free</code> : memory is never returned; program grows until it crashes
Use after free	Accessing memory after <code>free</code> : undefined behaviour
Double free	Calling <code>free</code> twice on the same pointer: corrupts the heap
Buffer overflow	Writing past the end of allocated memory: overwrites other data

These bugs are hard to debug and are the source of many real-world security vulnerabilities. **System Programming** teaches tools (Valgrind, AddressSanitizer) to detect them.

Where You Will See This Again

- **System Programming:** `malloc`, `calloc`, `realloc`, `free`; the heap; memory-safe patterns.
- **Data Structures:** every linked list, tree, and hash table uses dynamic allocation for each node.
- **C++ / Modern Languages:** `new` / `delete` in C++; garbage collectors in Java/Python do this automatically.